



PROGRAMACION COMPETITIVA

Notebook

Python

Algorithms • Data Structures • Mathematics
Graphs • Dynamic Programming • Strings
Geometry • Flows

Reference Notebook for
ICPC • Codeforces • UVA
CSES • Gym

Compiled and maintained by
David Santiago García Preciado
Systems Engineering
Universidad El Bosque

Version 1.0
Last Updated
July 30, 2026

Contents

1	Entrada/Salida Rápida en Python	2			
1.1	Lectura línea a línea (caso general)	2			
1.2	Lectura total + tokenización (máxima velocidad)	2			
1.3	Casos con bandera (condicional)	2			
1.4	Casos hasta EOF (sin contador T)	2			
1.5	Salida rápida	2			
2	Herramientas y Funciones Esenciales de Python	3			
2.1	Iteradores y funciones integradas	3			
2.2	Expresiones Regulares (módulo re)	4			
2.3	Diccionarios: dict, defaultdict y Counter	5			
2.4	Manipulación de cadenas	7			
2.5	Módulos matemáticos	8			
2.6	Librerías estándar útiles para ICPC	10			
3	Estructuras de Datos	13			
3.1	Fenwick Tree (Árbol de Índices Binarios / BIT)	13			
3.2	Suma de Prefijos con Mapa de Primera Aparición	13			
3.3	Fenwick Tree con XOR (Nim por Rangos, Sprague-Grundy)	14			
4	Algoritmos de Grafos	16			
4.1	Emparejamiento Bipartito Máximo (Algoritmo de Kuhn)	16			
4.2	Emparejamiento Bipartito por Segmentación de Filas/Columnas	16			
4.3	Cobertura Mínima de Vértices en Grafos Bipartitos (Teorema de König)	17			
4.4	Componentes Fuertemente Conexas (Algoritmo de Kosaraju)	18			
4.5	Circuito Euleriano (Algoritmo de Hierholzer)	19			
4.6	Puntos de Articulación (Algoritmo de Tarjan)	19			
4.7	Algoritmo de Bron-Kerbosch (Cliques Máximos)	20			
5	Búsqueda y Optimización	22			
5.1	Búsqueda Binaria sobre la Respuesta + Two Pointers	22			
5.2	Enumeración por Máscaras de Bits (Bitmask Brute Force)	22			
5.3	Búsqueda Binaria sobre la Respuesta con Ventana de Bonificación	23			
6	Matemáticas y Simulación	24			
6.1	Patrón de Reducción Binaria (tipo Josefo)	24			
6.2	Fuerza Bruta / Simulación Directa	24			
6.3	Simulación Lineal (Máximo de Prefijo)	24			
6.4	Simulación de Eventos con Colas	25			
6.5	Simulación XOR con Detección de Ciclos	26			
7	Construcción y Combinatoria	27			
7.1	Algoritmos Constructivos: Invariantes y Paridad	27			
7.2	Algoritmos Constructivos: Construcción Explícita	27			
7.3	Combinatoria: Conteo por Simetría de Binomios	28			
7.4	Construcción Greedy por Alternancia (Límites de Racha Consecutiva)	28			
7.5	Distribución de Días de Espera (Stars and Bars)	29			
8	Algoritmos de Cadenas (Strings)	30			
8.1	Parejas Incompatibles en Secuencias de Paréntesis (Bloques de Nivel Superior)	30			
8.2	Rotación Mínima de un String (Algoritmo de Booth)	30			
8.3	Validación de Matrioshkas Generalizadas (Pila con Restricción de Capacidad)	31			
8.4	Normalización y Verificación de Palíndromos	32			
8.5	Eliminación de Ciclos en una Ruta (Stack Reduction)	32			

Este Notebook tiene una estructura simple. Hay preguntas guía para reconocer cuándo aplica el algoritmo, para qué sirve, su complejidad, el fundamento teórico, y un ejercicio real resuelto con esa técnica.

1 Entrada/Salida Rápida en Python

En Python el I/O es el cuello de botella más común en ICPC. Estas son las 4 situaciones típicas y el patrón recomendado para cada una.

1.1 Lectura línea a línea (caso general)

Cuando cada línea tiene una cantidad de datos conocida y el número de líneas no es extremo (hasta 10^5). Es el patrón por defecto.

```
import sys
input = sys.stdin.buffer.readline
n = int(input())
arr = list(map(int, input().split()))
```

1.2 Lectura total + tokenización (máxima velocidad)

Cuando hay muchísimos números sueltos (10^5 – 10^6 +) sin importar en cuántas líneas están repartidos. Es la forma más rápida en Python, ya que es una sola llamada al sistema.

```
import sys
data = sys.stdin.buffer.read().split()
idx = 0 # el idx es lo que recorre el arreglo de tokens
n = int(data[idx]); idx += 1
arr = list(map(int, data[idx:idx + n])); idx += n
```

1.3 Casos con bandera (condicional)

El input termina cuando aparece un valor en concreto. No se sabe de antemano cuántos casos hay.

```
import sys
input = sys.stdin.buffer.readline
while True:
    a, b = map(int, input().split())
    if a == 0 and b == 0: # en este caso el valor
        concreto es "0 0"
        break
    # aqui se procesan los casos
```

1.4 Casos hasta EOF (sin contador T)

No hay número de casos declarado, por lo que se procesa hasta que no haya mas inputs.

```
import sys
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    # se procesa el caso
```

Una alternativa con los casos infinitos es usar el metodo de tokenización.

```
import sys
data = sys.stdin.buffer.read().split()
idx = 0
while idx < len(data):
    # se procesa el caso
```

1.5 Salida rápida

Nunca usar print() y mejor acumular resultados en una lista y escribir todo de una sola vez.

```
ans = []
# ans.append(result) dentro del loop
sys.stdout.write("\n".join(map(str, ans)) + "\n")
```

2 Herramientas y Funciones Esenciales de Python

2.1 Iteradores y funciones integradas

Estas funciones permiten recorrer estructuras de datos, transformar secuencias y escribir código más corto y legible sin sacrificar rendimiento.

2.1.1 enumerate()

Recorrer un iterable obteniendo simultáneamente el índice y el elemento, evitando mantener un contador manual.

```
arr = [5, 8, 2]
for i, x in enumerate(arr):
    print(i, x)
for i, x in enumerate(arr, 1):    # índices comienzan en 1
    print(i, x)
```

Es especialmente útil cuando el problema utiliza índices 1-indexados.

2.1.2 zip()

Recorrer múltiples iterables en paralelo. El recorrido termina cuando el iterable más corto finaliza.

```
a = [1, 2, 3]
b = [4, 5, 6]
for x, y in zip(a, b):
    print(x, y)

pares = list(zip(a, b))
# [(1, 4), (2, 5), (3, 6)]
```

También puede utilizarse para desempaquetar listas de tuplas.

```
pares = [(1, 4), (2, 5), (3, 6)]
a, b = zip(*pares)
print(a); print(b)
```

2.1.3 iter() y next()

iter() crea un iterador sobre cualquier iterable. next() consume el siguiente elemento del iterador. Se suelen usar cuando se procesan tokens manualmente.

```
data = ["5", "10", "20", "30"]
it = iter(data)
n = int(next(it))
arr = [int(next(it)) for _ in range(n - 1)]
```

También puede especificarse un valor por defecto.

```
x = next(it, None)
```

2.1.4 any() y all()

Evaluar rápidamente condiciones sobre un iterable.

any() devuelve True si al menos un elemento cumple la condición. all() devuelve True únicamente si todos los elementos la cumplen. Ambas realizan *short-circuit*, por lo que dejan de recorrer el iterable tan pronto conocen la respuesta.

```
if any(x % 2 == 0 for x in arr):
    print("Existe un numero par")
if all(x > 0 for x in arr):
    print("Todos son positivos")
```

2.1.5 map()

Aplica una función a todos los elementos de un iterable. Es muy utilizado durante la lectura de entrada.

```
nums = list(map(int, input().split()))
cuadrados = list(map(lambda x: x * x, nums))
```

También puede utilizarse con funciones propias.

```
def f(x):
    return x + 1
nuevo = list(map(f, nums))
```

2.1.6 sorted()

Devuelve una nueva lista ordenada sin modificar la original.

Puede ordenarse mediante una función key.

```
nums = sorted(nums)
nums = sorted(nums, reverse=True)
personas = sorted(personas, key=lambda x: x.edad)
pares = sorted(freq.items(), key=lambda kv: -kv[1])
```

Complejidad: $\mathcal{O}(n \log n)$

Python utiliza Timsort, un algoritmo estable muy eficiente para datos parcialmente ordenados.

2.1.7 reversed()

Permite recorrer una secuencia en orden inverso sin crear una copia.

```
for x in reversed(arr):
    print(x)
```

Es preferible a lo siguiente, que crea una copia de la lista:

```
arr[::-1]
```

2.1.8 encode() y decode()

Aunque rara vez aparecen en ICPC, pueden resultar útiles cuando un problema involucra archivos, comunicación o codificaciones específicas.

encode() convierte un string en bytes. decode() realiza la operación inversa.

```
s = "ICPC"
b = s.encode("utf-8")
print(b)
# b'ICPC'
texto = b.decode("utf-8")
print(texto)
# ICPC
```

Las codificaciones más comunes:

- utf-8
- ascii

- latin-1

En programación competitiva normalmente toda la entrada ya llega como texto UTF-8, por lo que estas funciones aparecen con poca frecuencia.

Resumen

Función	Uso principal
<code>enumerate()</code>	Recorrer con índice
<code>zip()</code>	Recorrer varios iterables
<code>iter()</code>	Crear un iterador
<code>next()</code>	Consumir siguiente elemento
<code>any()</code>	Existe algún elemento que cumpla
<code>all()</code>	Todos cumplen
<code>map()</code>	Aplicar función a todos los elementos
<code>sorted()</code>	Ordenar sin modificar el original
<code>reversed()</code>	Recorrer en orden inverso
<code>encode()</code>	String → bytes
<code>decode()</code>	Bytes → string

2.2 Expresiones Regulares (módulo re)

El módulo `re` permite buscar, validar, reemplazar y extraer información de cadenas mediante patrones.

En programación competitiva suele utilizarse cuando la entrada posee un formato irregular, contiene separadores complejos o es necesario validar cadenas.

```
import re
```

2.2.1 Funciones principales

Función	Descripción
<code>re.match()</code>	Busca únicamente al inicio del string.
<code>re.search()</code>	Busca la 1ra coincidencia en cualquier posición.
<code>re.findall()</code>	Devuelve todas las coincidencias en una lista.
<code>re.finditer()</code>	Devuelve un iter con todos los matches.
<code>re.fullmatch()</code>	Verifica que toda la cadena cumpla patrón.
<code>re.sub()</code>	Reemplaza coincidencias.
<code>re.split()</code>	Divide una cadena utilizando un patrón.
<code>re.compile()</code>	Compila una regex para reutilizarla.

2.2.2 Símbolos más utilizados

Patrón	Significado
<code>.</code>	Cualquier carácter excepto salto de línea
<code>\d</code>	Dígito
<code>\D</code>	No dígito
<code>\w</code>	Letra, número o <code>_</code>
<code>\W</code>	No alfanumérico
<code>\s</code>	Espacio en blanco
<code>\S</code>	No espacio
<code>+</code>	Una o más repeticiones
<code>*</code>	Cero o más repeticiones
<code>?</code>	Cero o una repetición
<code>{n}</code>	Exactamente n repeticiones
<code>{n,m}</code>	Entre n y m repeticiones
<code>[]</code>	Conjunto de caracteres
<code>[^]</code>	Negación del conjunto
<code>()</code>	Grupo de captura
<code>(?:)</code>	Grupo sin captura
<code> </code>	OR lógico
<code>^</code>	Inicio del string
<code>\$</code>	Final del string

2.2.3 Ejemplos básicos

```
import re
nums = re.findall(r'~?\d+', s)
nums = list(map(int, nums))
tokens = re.split(r'\s+', s.strip())
m = re.match(r'^(\d+)-(\d+)$', s)
if m:
    a = int(m.group(1))
    b = int(m.group(2))
limpio = re.sub(r'[^a-zA-Z0-9]', '', s)
```

2.2.4 Patrones útiles para ICPC

Patrón	Detecta
<code>\d+</code>	Enteros positivos
<code>-?\d+</code>	Enteros con signo
<code>\d+\.\d+</code>	Números decimales
<code>[A-Za-z]+</code>	Palabras
<code>[A-Za-z0-9]+</code>	Alfanuméricos
<code>[aeiou]</code>	Vocales
<code>[^aeiou]</code>	Todo excepto vocales
<code>\s+</code>	Uno o más espacios
<code>\w+</code>	Identificadores
<code>.+</code>	Cualquier texto

2.2.5 Combinaciones frecuentes

Extraer todos los enteros.

```
re.findall(r'~?\d+', s)
```

Extraer únicamente números positivos.

```
re.findall(r'\d+', s)
```

Extraer palabras.

```
re.findall(r'[A-Za-z]+', s)
```

Extraer palabras y números.

```
re.findall(r'\w+', s)
```

Separar por múltiples espacios.

```
re.split(r'\s+', s.strip())
```

Eliminar caracteres especiales.

```
re.sub(r'[^\A-Za-z0-9]', '', s)
```

Eliminar espacios repetidos.

```
re.sub(r'\s+', ' ', s)
```

Validar un correo electrónico sencillo.

```
re.fullmatch(r'\w+@\w+\.\w+', s)
```

Validar una fecha DD/MM/AAAA.

```
re.fullmatch(r'\d{2}/\d{2}/\d{4}', s)
```

Validar un entero.

```
re.fullmatch(r'-?\d+', s)
```

Buscar únicamente al inicio.

```
re.match(r'ICPC', s)
```

Buscar en cualquier posición.

```
re.search(r'ICPC', s)
```

2.2.6 Compilar una expresión regular

Cuando un mismo patrón será utilizado muchas veces, es recomendable compilarlo una única vez. Esto evita recompilar la expresión regular en cada llamada.

```
pat = re.compile(r'-?\d+')
for linea in texto:
    nums = pat.findall(linea)
```

2.2.7 Complejidad

La mayoría de operaciones tienen complejidad aproximadamente $\mathcal{O}(n)$ respecto al tamaño del texto, aunque algunos patrones muy complejos pueden provocar retroceso (*backtracking*) y aumentar considerablemente el tiempo de ejecución.

En problemas de ICPC normalmente se utilizan patrones simples, por lo que el rendimiento suele ser lineal.

Resumen

Función	Uso principal
match	Buscar al inicio
search	Buscar primera coincidencia
findall	Obtener todas las coincidencias
finditer	Iterar sobre coincidencias
fullmatch	Validar una cadena completa
split	Separar mediante un patrón
sub	Reemplazar texto
compile	Reutilizar un patrón

2.3 Diccionarios: dict, defaultdict y Counter

Los diccionarios son una de las estructuras de datos más utilizadas en programación competitiva. Permiten almacenar pares **clave** : **valor** con acceso promedio en tiempo constante.

En Python, la implementación utiliza una tabla hash, por lo que la mayoría de operaciones tienen complejidad promedio $\mathcal{O}(1)$ aunque en el peor caso pueden degradarse a $\mathcal{O}(n)$

si existen muchas colisiones.

2.3.1 Crear un diccionario

```
freq = {}
persona = {"nombre": "David", "edad": 20}
```

2.3.2 Acceso y recuperación

Acceder directamente mediante corchetes genera un **KeyError** si la clave no existe.

```
x = freq["a"]
```

Por ello normalmente es preferible utilizar `get()`.

```
freq.get("a")
freq.get("a", 0)
freq.get("a", [])
```

Las funciones principales de recuperación son:

Función	Descripción
get()	Obtiene un valor sin lanzar excepción.
keys()	Devuelve todas las claves.
values()	Devuelve todos los valores.
items()	Devuelve pares (clave, valor).

Ejemplos:

```
for k in freq.keys():
    ...
for v in freq.values():
    ...
for k, v in freq.items():
    ...
```

2.3.3 Modificar un diccionario

Insertar o modificar una clave.

```
freq["a"] = 5
```

Actualizar múltiples valores.

```
a = {"x":1}; b = {"y":2}  
a.update(b)
```

Crear una clave únicamente si no existe.

```
freq.setdefault("python", [])  
freq["python"].append(10)
```

2.3.4 Eliminar elementos

Eliminar una clave.

```
freq.pop("a")
```

Eliminar devolviendo un valor por defecto.

```
freq.pop("a", 0)
```

Eliminar el último elemento insertado.

```
freq.popitem()
```

Vaciar completamente el diccionario.

```
freq.clear()
```

2.3.5 Funciones integradas

Cantidad de elementos.

```
len(freq)
```

Copiar un diccionario.

```
nuevo = freq.copy()
```

Comprobar existencia.

```
if "python" in freq:  
    ...
```

Eliminar una copia sin modificar el original.

```
nuevo = dict(freq)
```

2.3.6 Ordenar un diccionario

Ordenar por clave.

```
sorted(freq.items())
```

Ordenar por valor.

```
sorted(freq.items(), key=lambda kv: kv[1])
```

Orden descendente.

```
sorted(freq.items(), key=lambda kv: -kv[1])
```

2.3.7 defaultdict

Normalmente antes de incrementar una frecuencia se necesita comprobar si la clave existe.

```
freq = {}  
freq[x] = freq.get(x, 0) + 1
```

Con defaultdict esto desaparece.

```
from collections import defaultdict  
  
freq = defaultdict(int)  
  
freq[x] += 1
```

También puede utilizarse para listas.

```
grupos = defaultdict(list)  
for x in arr:  
    grupos[x % 3].append(x)
```

Con conjuntos.

```
adj = defaultdict(set)  
adj[u].add(v)
```

Y con cualquier función.

```
from collections import defaultdict  
d = defaultdict(lambda: -1)
```

2.3.8 Counter

Cuando únicamente interesa contar frecuencias, la clase Counter simplifica mucho el código.

```
from collections import Counter  
freq = Counter(arr)
```

Obtener el elemento más frecuente.

```
freq.most_common(1)
```

Los tres más frecuentes.

```
freq.most_common(3)
```

Frecuencia de un elemento.

```
freq[x]
```

2.3.9 Aplicaciones frecuentes en ICPC

Contar frecuencias.

```
freq = defaultdict(int)  
for x in arr:  
    freq[x] += 1
```

Agrupar elementos.

```
g = defaultdict(list)  
for x in arr:  
    g[x % k].append(x)
```

Construir listas de adyacencia.

```
adj = defaultdict(list)  
for u, v in edges:  
    adj[u].append(v)
```

Construir grafos sin comprobar existencia.

```
adj = defaultdict(set)
adj[u].add(v)
```

Contar caracteres.

```
Counter(s)
```

Resumen

Función	Uso principal
get()	Obtener valor sin KeyError
keys()	Recorrer claves
values()	Recorrer valores
items()	Recorrer clave y valor
update()	Actualizar con otro diccionario
setdefault()	Crear si no existe
pop()	Eliminar una clave
popitem()	Eliminar último elemento
clear()	Vaciar el diccionario
copy()	Copia superficial
len()	Número de elementos
defaultdict	Valor por defecto automático
Counter	Conteo automático de frecuencias

2.4 Manipulación de cadenas

Las cadenas (`str`) son inmutables en Python, por lo que todas las operaciones que modifican su contenido devuelven una nueva cadena.

La mayoría de problemas de parsing, simulación y procesamiento de texto utilizan estas funciones.

2.4.1 replace()

Reemplaza ocurrencias de una subcadena.

```
s = "banana"
s.replace("a", "o")
# "bonono"
s.replace("a", "o", 2)
# "bonona"
```

Complejidad promedio: $\mathcal{O}(n)$

2.4.2 split()

Divide una cadena utilizando un separador.

```
s.split()
s.split(",")
s.split(":")
s.split("\n")
```

Si no se especifica un separador, utiliza cualquier cantidad de espacios consecutivos.

```
s = "a b c"
print(s.split())
# ['a', 'b', 'c']
```

2.4.3 join()

Une una colección de cadenas utilizando un separador.

Es mucho más eficiente que concatenar strings dentro de un ciclo.

```
nums = ["1", "2", "3"]
", ".join(nums)
# "1, 2, 3"
" ".join(nums)
# "1 2 3"
```

Complejidad: $\mathcal{O}(n)$

2.4.4 strip(), lstrip() y rstrip()

Eliminan caracteres al inicio y/o final.

```
s.strip()
s.lstrip()
s.rstrip()
```

También pueden eliminar caracteres específicos.

```
--ICPC--".strip("-")
# ICPC
```

2.4.5 find() y rfind()

Buscan una subcadena.

Devuelven

- la posición encontrada;
- 1 si no existe.

```
s.find("abc")
s.rfind("abc")
```

2.4.6 count()

Cuenta cuántas veces aparece una subcadena.

```
s.count("ab")
```

2.4.7 startswith() y endswith()

Permiten verificar prefijos y sufijos.

```
s.startswith("abc")
s.endswith(".cpp")
```

Muy útiles para parsing.

2.4.8 lower(), upper() y capitalize()

Cambian el formato de las letras.

```
s.lower()
s.upper()
s.capitalize()
```

2.4.9 isalpha(), isdigit(), isalnum()

Verifican el contenido de una cadena.

```
s.isalpha()
s.isdigit()
s.isalnum()
s.isspace()
```

2.4.10 partition()

Divide únicamente en la primera aparición del separador.

```
s.partition("=")
# ('x', '=', '15')
```

A diferencia de `split()`, siempre devuelve exactamente tres elementos.

2.4.11 Concatenación

```
a + b
a += b
```

Cuando se concatenan muchas cadenas es preferible utilizar

```
"".join(lista)
```

ya que evita crear múltiples copias intermedias.

2.4.12 Formateo de cadenas

Python permite insertar variables directamente mediante f-strings.

```
x = 3.14159265
f"{x:.2f}"
# 3.14
f"{x:.10f}"
# 3.1415926500
nombre = "ICPC"
f"Hola {nombre}"
```

Este mecanismo es el recomendado para imprimir números reales.

2.4.13 Aplicaciones frecuentes en ICPC

Eliminar espacios al leer una línea.

```
s = input().strip()
```

Separar una línea.

```
tokens = input().split()
```

Unir la respuesta.

```
print(" ".join(ans))
```

Comprobar un prefijo.

```
if s.startswith("YES"):
    ...
```

Eliminar caracteres.

```
s = s.replace(" ", "")
```

Buscar una palabra.

```
if s.find("abc") != -1:
    ...
```

Contar caracteres.

```
vocales = s.count("a")
```

Resumen

Función	Uso principal
<code>replace()</code>	Reemplazar texto
<code>split()</code>	Dividir una cadena
<code>join()</code>	Unir cadenas
<code>strip()</code>	Eliminar espacios
<code>find()</code>	Buscar subcadena
<code>rfind()</code>	Buscar desde el final
<code>count()</code>	Contar ocurrencias
<code>startswith()</code>	Verificar prefijo
<code>endswith()</code>	Verificar sufijo
<code>lower()</code>	Minúsculas
<code>upper()</code>	Mayúsculas
<code>capitalize()</code>	Primera letra mayúscula
<code>isalpha()</code>	Solo letras
<code>isdigit()</code>	Solo dígitos
<code>isalnum()</code>	Alfanumérico
<code>isspace()</code>	Solo espacios
<code>partition()</code>	Dividir una vez

2.5 Módulos matemáticos

Python incluye varios módulos que simplifican operaciones matemáticas frecuentes en programación competitiva.

El más utilizado es `math`.

```
import math
```

2.5.1 Redondeo

Función	Descripción
<code>round(x,n)</code>	Redondea a n decimales
<code>math.floor(x)</code>	Entero menor o igual
<code>math.ceil(x)</code>	Entero mayor o igual
<code>math.trunc(x)</code>	Elimina la parte decimal

```
import math
math.floor(3.8) # 3
math.ceil(3.2) # 4
round(3.141592,2) # 3.14
math.trunc(7.98) # 7
factor = 10 ** N
floor_n = math.floor(x * factor) / factor
ceil_n = math.ceil(x * factor) / factor
```

2.5.2 Valor absoluto

```
abs(x)
math.fabs(x)
```

Normalmente basta utilizar

```
abs(x)
```

2.5.3 Raíces

Función	Uso
<code>math.sqrt(x)</code>	Raíz cuadrada
<code>math.isqrt(x)</code>	Raíz entera exacta

```
math.sqrt(25) # 5.0
math.isqrt(30) # 5
```

En teoría de números suele preferirse `isqrt()`.

2.5.4 Máximo común divisor y mínimo común múltiplo

```
math.gcd(a,b)
math.lcm(a,b)
```

También aceptan más de dos argumentos.

```
math.gcd(a,b,c)
math.lcm(a,b,c)
```

Complejidad: $\mathcal{O}(\log(\min(a,b)))$

2.5.5 Potencias

```
pow(a,b)
pow(a,b,m)
a ** b
math.pow(a,b)
```

Observaciones.

- `pow(a,b,m)` calcula la potencia módulo `m` de forma eficiente.
- `**` devuelve enteros cuando es posible.
- `math.pow()` siempre devuelve float.

2.5.6 Factoriales

```
math.factorial(n)
```

Complejidad: $\mathcal{O}(n)$

2.5.7 Combinatoria

Desde Python 3.8 existen funciones incorporadas.

```
math.comb(n,k)
math.perm(n,k)
```

Ejemplos.

```
math.comb(5,2) # 10
math.perm(5,2) # 20
```

2.5.8 Logaritmos

```
math.log(x)
math.log2(x)
math.log10(x)
```

2.5.9 Trigonometría

```
math.sin(x)
math.cos(x)
math.tan(x)
math.asin(x)
math.acos(x)
```

```
math.atan(x)
```

Los ángulos siempre se expresan en radianes.

2.5.10 Distancias

```
math.dist(p1,p2)
math.hypot(x,y)
```

```
math.hypot(3,4) # 5
```

2.5.11 Producto

```
math.prod(lista)
```

Ejemplo.

```
math.prod([2,3,4]) # 24
```

2.5.12 Constantes

```
math.pi
math.e
math.inf
math.nan
```

2.5.13 Fracciones exactas

Cuando los números en punto flotante producen errores de precisión puede utilizarse el módulo

```
from fractions import Fraction
```

```
Fraction(1,3)
Fraction(1,6)
Fraction(1,3)+Fraction(1,6)
```

Resultado.

```
Fraction(1,2)
```

No existe error de redondeo.

2.5.14 División entera

En Python la división entera siempre redondea hacia menos infinito.

```
7 // 2
# 3
-7 // 2
# -4
```

Este comportamiento es diferente al de C++ y Java.

2.5.15 Aplicaciones frecuentes en ICPC

Máximo común divisor.

```
math.gcd(a,b)
```

Potencia modular.

```
pow(a,b,MOD)
```

Raíz entera.

```
math.isqrt(n)
```

Binomiales.

```
math.comb(n,k)
```

Permutaciones.

```
math.perm(n,k)
```

Fracciones exactas.

```
Fraction(a,b)
```

Resumen

Función	Uso principal
<code>floor()</code>	Redondear hacia abajo
<code>ceil()</code>	Redondear hacia arriba
<code>trunc()</code>	Eliminar parte decimal
<code>gcd()</code>	Máximo común divisor
<code>lcm()</code>	Mínimo común múltiplo
<code>sqrt()</code>	Raíz cuadrada
<code>isqrt()</code>	Raíz entera
<code>factorial()</code>	Factorial
<code>comb()</code>	Combinaciones
<code>perm()</code>	Permutaciones
<code>pow()</code>	Potencias
<code>log2()</code>	Logaritmo base 2
<code>hypot()</code>	Distancia euclidiana
<code>prod()</code>	Producto de un iterable
<code>Fraction</code>	Fracciones exactas

2.6 Librerías estándar útiles para ICPC

Python posee una biblioteca estándar muy completa. Conocer estos módulos permite resolver muchos problemas sin implementar estructuras o algoritmos desde cero.

2.6.1 collections

Contiene estructuras de datos optimizadas.

```
from collections import deque
from collections import defaultdict
from collections import Counter
from collections import OrderedDict
from collections import namedtuple
```

Clase	Uso
<code>deque</code>	Cola doble, BFS, Sliding Window
<code>defaultdict</code>	Dict con valor por defecto
<code>Counter</code>	Conteo automático de frecuencias
<code>OrderedDict</code>	Dict que conserva operaciones de orden
<code>namedtuple</code>	Tuplas con nombres para los campos

a) *deque*

Permite insertar y eliminar elementos por ambos extremos en $\mathcal{O}(1)$

```
from collections import deque
```

```
q = deque()
q.append(5)
q.appendleft(2)
q.pop()
q.popleft()
q.rotate(1)
q.rotate(-1)
```

Es la estructura estándar para implementar BFS.

b) *defaultdict*

Ya estudiado en la sección anterior.

Se utiliza principalmente para:

- frecuencias;
- listas de adyacencia;
- agrupaciones.

c) *Counter*

Muy útil para conteo de frecuencias.

```
freq = Counter(arr)
freq.most_common(3)
```

2.6.2 heapq

Implementa un montículo mínimo (Min Heap).

```
import heapq
```

Funciones principales.

Función	Uso
<code>heappush()</code>	Insertar
<code>heappop()</code>	Extraer mínimo
<code>heapify()</code>	Construir heap
<code>heapreplace()</code>	Reemplazar mínimo
<code>nlargest()</code>	Mayores elementos
<code>nsmallest()</code>	Menores elementos

Ejemplo.

```
heap=[]
heapq.heappush(heap,4)
heapq.heappush(heap,2)
heapq.heappush(heap,8)
heapq.heappop(heap)
```

Todas las operaciones de inserción y extracción tienen complejidad $\mathcal{O}(\log n)$

Se utiliza principalmente en:

- Dijkstra;
- Prim;
- Huffman;
- Top-K.

2.6.3 bisect

Permite realizar búsqueda binaria sobre listas ordenadas.

```
from bisect import bisect_left
from bisect import bisect_right
from bisect import insort
```

```
bisect_left(arr,x)
bisect_right(arr,x)
insort(arr,x)
```

Todas las búsquedas tienen complejidad $\mathcal{O}(\log n)$

2.6.4 functools

Funciones de orden superior.

```
from functools import lru_cache
from functools import cache
from functools import cmp_to_key
from functools import reduce
```

a) *lru_cache*

Muy utilizado para memoización.

```
@lru_cache(None)
def dp(i,j):
    ...
```

b) *cmp_to_key*

Permite utilizar comparadores personalizados.

```
sorted(arr,key=cmp_to_key(cmp))
```

2.6.5 itertools

Herramientas para trabajar con iteradores.

```
from itertools import permutations
from itertools import combinations
from itertools import combinations_with_replacement
from itertools import product
from itertools import accumulate
from itertools import cycle
from itertools import repeat
from itertools import pairwise
```

Función	Uso
permutations	Permutaciones
combinations	Combinaciones
product	Producto cartesiano
accumulate	Prefijos acumulados
cycle	Ciclo infinito
repeat	Repetición
pairwise	Pares consecutivos

Ejemplos.

```
permutations(arr)
combinations(arr,2)
product(a,b)
accumulate(arr)
```

2.6.6 operator

Contiene operadores como funciones.

```
from operator import itemgetter
from operator import attrgetter
```

Ejemplo.

```
sorted(arr,key=itemgetter(1))
```

2.6.7 decimal

Permite realizar aritmética decimal exacta.

```
from decimal import Decimal
```

Normalmente no se utiliza en ICPC debido a su menor rendimiento.

2.6.8 fractions

Permite trabajar con fracciones exactas.

```
from fractions import Fraction
```

Muy útil cuando se desea evitar errores de punto flotante.

2.6.9 array

Implementa arreglos homogéneos.

```
from array import array
```

Consumen menos memoria que una lista convencional.

2.6.10 queue

```
from queue import PriorityQueue
```

Aunque implementa colas de prioridad, en programación competitiva normalmente se prefiere utilizar

```
heapq
```

por ser considerablemente más rápido.

2.6.11 sys

El módulo más utilizado para entrada y salida rápida.

```
import sys

input = sys.stdin.readline

sys.setrecursionlimit(200000)
```

2.6.12 Aplicaciones frecuentes en ICPC

Problema	Librería recomendada
BFS	deque
Dijkstra	heapq
Memoización	lru_cache
Conteo	Counter
Frecuencias	defaultdict
Binary Search	bisect
Permutaciones	itertools
Combinaciones	itertools
Ordenamientos personalizados	cmp_to_key
Prefijos	accumulate

Resumen

Módulo	Uso principal
<code>collections</code>	Estructuras de datos
<code>heapq</code>	Heap mínimo
<code>bisect</code>	Búsqueda binaria
<code>functools</code>	Memoización y comparadores
<code>itertools</code>	Iteradores avanzados
<code>operator</code>	Funciones auxiliares para ordenar
<code>fractions</code>	Fracciones exactas
<code>decimal</code>	Decimales exactos
<code>array</code>	Arreglos homogéneos
<code>queue</code>	Colas sincronizadas
<code>sys</code>	Entrada rápida y recursión

3 Estructuras de Datos

3.1 Fenwick Tree (Árbol de Índices Binarios / BIT)

Preguntas guía:

- ¿Necesito sumas o conteos acumulados sobre un rango que cambia con actualizaciones puntuales?
- ¿Los updates son de un solo índice y las queries son sobre prefijos o rangos $[l, r]$?
- ¿Si no hay updates "online", puedo resolver offline ordenando las queries (p. ej. por su extremo derecho)?

Utilidad: Mantener sumas de prefijo (o cualquier operación invertible) sobre un arreglo con actualizaciones puntuales, respondiendo consultas de rango en tiempo logarítmico. Es la base de técnicas como conteo de elementos distintos por rango, inversiones en un arreglo y frecuencias acumuladas.

Complejidad: Construcción $\mathcal{O}(n)$. Update puntual $\mathcal{O}(\log n)$. Query de prefijo $\mathcal{O}(\log n)$. Espacio $\mathcal{O}(n)$.

Fundamento teórico: Cada posición i del arreglo interno almacena la suma de un bloque de tamaño $(i \& -i)$, el bit menos significativo de i . Al actualizar se "sube" sumando $(i \& -i)$; al consultar un prefijo se "baja" restando $(i \& -i)$. Esto aprovecha la representación binaria de los índices para cubrir cualquier prefijo con solo $\mathcal{O}(\log n)$ bloques.

Ejercicio aplicado

Dado un arreglo de n números y q consultas (i, j) , responder para cada una cuántos valores distintos hay en el subarreglo $[i, j]$. Técnica: procesar las consultas offline ordenadas por extremo derecho r , recorriendo el arreglo de izquierda a derecha y manteniendo en el BIT un "1" solo en la última aparición de cada valor.

I/O: $n \leq 30\,000$ y $q \leq 200\,000$ hasta $\sim 430\,000$ tokens de entrada: se usa lectura total + tokenización (patrón 2) en vez de readline por línea.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.readline
    n = int(data())
    nums = list(map(int, data().split()))
    q = int(data())

    queries = []
    for i in range(q):
        l, r = map(int, data().split())
        queries.append((r, l, i))  # se ordena por r
    queries.sort()

    bit = [0] * (n + 1)

    def update(i, val):
        while i <= n:
            bit[i] += val
            i += i & (-i)  # ultimo bit encendido

    def query(i):
        res = 0
```

```
while i > 0:
    res += bit[i]
    i -= i & (-i)
return res

last = {}  # ultima posicion
# vista de cada valor
ans = [0] * q
cur = 0
for r, l, qi in queries:
    while cur < r:  # avanza el
        puntero hasta r
        cur += 1
        x = nums[cur - 1]
        prev = last.get(x)
        if prev:
            update(prev, -1)  # desactiva la
        # aparicion anterior
        update(cur, 1)  # activa la
        # aparicion actual
        last[x] = cur
    ans[qi] = query(r) - query(l - 1)

sys.stdout.write("\n".join(map(str, ans)))

solve()
```

3.2 Suma de Prefijos con Mapa de Primera Aparición

Preguntas guía:

- ¿Piden la subsecuencia contigua más larga que cumple una condición de "balance" (mismo número de dos tipos de elementos, o suma total igual a un valor fijo)?
- ¿Puedo transformar cada elemento en un delta $(+1/-1/0)$ tal que la condición se traduzca en "suma de un subarreglo == 0"?
- ¿Me alcanza con guardar, para cada valor de suma acumulada, solo la PRIMERA posición donde apareció?

¿Para qué sirve? Encontrar en $\mathcal{O}(n)$ el subarreglo contiguo más largo cuya suma (o balance entre dos tipos de elementos) es igual a un valor fijo, sin revisar los $\mathcal{O}(n^2)$ subarreglos posibles.

Complejidad: $\mathcal{O}(n)$ tiempo, $\mathcal{O}(n)$ espacio.

Fundamento teórico: Si $P[i]$ es la suma acumulada de los primeros i elementos (con la transformación adecuada), la suma del subarreglo $(i+1..j)$ es $P[j] - P[i]$. Esa suma es 0 exactamente cuando $P[j] = P[i]$. Para maximizar la longitud $j-i$, basta con recordar, para cada valor de prefijo visto, la PRIMERA posición en que apareció (nunca la última), y comparar contra ella cada vez que el valor se repite.

Ejercicio aplicado

Dada una secuencia de N transacciones (positiva = depósito, negativa = retiro, cero = consulta), calcular el depósito máximo, el retiro más negativo, y la longitud del subperíodo contiguo más largo con igual cantidad de depósitos y retiros.

I/O: Múltiples casos con centinela "0" (patrón 3). $N \leq 10\,000 \rightarrow$ readline por línea (patrón 1); se probó también con lectura total y resultó más lenta para este patrón de

"un entero, luego un arreglo, repetido" (~2.5s vs ~3.9s en 500 casos de $N=10000$).

Código solución

```
import sys
input = sys.stdin.buffer.readline

def solve():
    out = []
    while True:
        n = int(input())
        if n == 0:
            break
        arr = list(map(int, input().split()))

        d = 0
        w = 0
        for x in arr:
            if x > 0:
                d = max(d, x)
            elif x < 0:
                w = min(w, x) # el mas negativo, no
                               # el de mayor magnitud

        # prefijo: +1 por deposito, -1 por retiro, 0 no
        # cambia nada
        first_seen = {0: 0} # valor de prefijo ->
        PRIMERA posicion vista
        prefix = 0
        r = 0
        for i, x in enumerate(arr, start=1):
            if x > 0:
                prefix += 1
            elif x < 0:
                prefix -= 1
            if prefix in first_seen:
                r = max(r, i - first_seen[prefix]) # se
                repite: hay balance
            else:
                first_seen[prefix] = i #
                primera vez: se guarda

        out.append(f"{d} {w} {r}")
        sys.stdout.write("\n".join(out) + "\n")
    solve()
```

3.3 Fenwick Tree con XOR (Nim por Rangos, Sprague-Grundy)

Preguntas guía:

- ¿El juego es una suma de sub-juegos independientes (aquí: columnas de fragmentos) donde en cada turno se modifica UNA sola pila?
- ¿Necesito responder "quién gana" para muchos rangos distintos, con actualizaciones puntuales intercaladas?
- ¿La operación de combinación relevante es XOR en vez de suma?

¿Para qué sirve? Generalizar el Fenwick Tree (visto para sumas) a CUALQUIER operación invertible para mantener el valor de Grundy combinado de un rango de pilas de Nim, con actualizaciones puntuales en tiempo logarítmico. Combinado con el teorema de Sprague-Grundy/Bouton, responde instantáneamente quién gana un juego de Nim sobre cualquier subrango.

Complejidad: Construcción $\mathcal{O}(n)$. Update puntual $\mathcal{O}(\log n)$. Query de rango $\mathcal{O}(\log n)$.

Fundamento teórico: En Nim con pilas independientes donde un jugador retira cualquier cantidad positiva de UNA pila, el valor de Grundy de una pila de tamaño v es exactamente v . Por el teorema de Bouton, el jugador que mueve PRIMERO gana si y solo si el XOR de todas las pilas en juego es distinto de 0. Como XOR es su propio inverso ($v \oplus v = 0$), el mismo esquema del Fenwick Tree que acumula sumas de prefijo funciona igual acumulando XOR de prefijo: para modificar el valor de una posición de v_{viejo} a v_{nuevo} , basta aplicar un update con $\Delta = v_{viejo} \oplus v_{nuevo}$ (equivalente a "quitar" el valor viejo y "poner" el nuevo en una sola operación).

Ejercicio aplicado

Juan y Frank juegan Nim sobre columnas de fragmentos: en cada turno, un jugador retira cualquier cantidad positiva de fragmentos de UNA columna; pierde quien no puede mover. Dadas Q consultas, cada una una "profecía" (determinar el ganador en un rango $[l, r]$, sabiendo que Frank siempre empieza) o un "ritual" (sumar x fragmentos a una columna), resolver cada profecía.

I/O: $N, Q \leq 10^6 \rightarrow$ lectura total con tokenización (patrón 2), y las funciones de update/query del Fenwick se escriben SIN llamadas a función (inline) para reducir el overhead de Python en este volumen de operaciones. Aun así, en el peor caso ($N=Q=10^6$) la solución pura en Python ronda ~7s si el juez es muy estricto con el límite de tiempo, este es el techo práctico de Python puro para este volumen sin cambiar de lenguaje.

Código solución

```
import sys

def main():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); q = int(data[idx + 1]); idx += 2
    arr = [0] * (n + 1)
    tree = [0] * (n + 1)

    # construccion en O(n): cada nodo se combina UNA vez
    # con su padre directo,
    # en vez de hacer n updates completos de O(log n)
    # cada uno
    for i in range(1, n + 1):
        v = int(data[idx]); idx += 1
        arr[i] = v
        tree[i] ^= v
        j = i + (i & (-i))
        if j <= n:
            tree[j] ^= tree[i]

    out = []
    out_append = out.append
    ANS = ("JUAN", "FRANK")
    P_BYTE = 80 # ord('P'): comparar el primer byte
                 # evita decodificar el token

    i2 = idx
    for _ in range(q):
        tok = data[i2]
        if tok[0] == P_BYTE:
            l = int(data[i2 + 1]); r = int(data[i2 + 2]);
            i2 += 3
            # xor(prefijo r) ^ xor(prefijo l-1) = xor del
            # rango [l, r]
```

```

    res = 0
    i = r
    while i:
        res ^= tree[i]
        i &= i - 1      # equivalente a i -=
i & (-i), un poco mas rapido
        i = 1 - i
        while i:
            res ^= tree[i]
            i &= i - 1
            out_append(ANS[res != 0])    # XOR != 0 ->
gana quien mueve primero (Frank)
        else:
            k = int(data[i2 + 1]); x = int(data[i2 + 2]);
            i2 += 3
            old = arr[k]
            new = old + x
            delta = old ^ new            # "sacar" old y "
poner" new en un solo update
            arr[k] = new
            i = k
            while i <= n:
                tree[i] ^= delta
                i += i & (-i)

    sys.stdout.write("\n".join(out) + "\n")

main()

```

4 Algoritmos de Grafos

4.1 Emparejamiento Bipartito Máximo (Algoritmo de Kuhn)

Preguntas guía:

- ¿Tengo dos conjuntos disjuntos (personas / tareas) y aristas que solo conectan un conjunto con el otro?
- ¿Me piden el número máximo de "parejas" o asignaciones simultáneas sin repetir elementos de ningún lado?
- ¿El tamaño de ambos lados permite $O(V \cdot E)$ (hasta unos pocos miles)?

Utilidad: Encontrar el máximo número de aristas seleccionables en un grafo bipartito sin que dos aristas compartan vértice (asignación óptima uno a uno). Ejemplos: asignar trabajadores a tareas, parejas de baile, cobertura de tareas.

Complejidad: $O(V \cdot E)$ con la versión simple de caminos aumentantes, donde V es el lado izquierdo y E el número de aristas.

Fundamento teórico: Se buscan repetidamente "caminos aumentantes": para cada vértice izquierdo se intenta, vía DFS, encontrar una pareja libre o "desalojar" a otro vértice ya emparejado para liberar espacio y aumentar el emparejamiento en 1. Es un caso particular del teorema de König / flujo máximo en redes.

Ejercicio aplicado

Hay n chicos y m chicas y k parejas de baile posibles. Hallar el número máximo de parejas simultáneas y mostrar una asignación válida.

I/O: $n, m \leq 500$ y $k \leq 1000 \rightarrow$ entrada pequeña, se usa readline por línea (patrón 1), suficiente y más simple de leer.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def try_kuhn(u, adj, matched, visited):
    # intenta emparejar el vertice u del lado izquierdo
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            # v esta libre, o su pareja actual puede reubicarse...
            if matched[v] == -1 or try_kuhn(matched[v],
            adj, matched, visited):
                matched[v] = u
                return True
    return False

def solve():
    n, m, k = map(int, input().split())
    adj = [[] for _ in range(n)]
    for _ in range(k):
        a, b = map(int, input().split())
        adj[a - 1].append(b - 1)

    matched = [-1] * m                # pareja actual
    total = 0                         # de cada vertice derecho
    for u in range(n):
        visited = [False] * m         # se reinicia en
        cada intento
```

```
        if try_kuhn(u, adj, matched, visited):
            total += 1

    out = [str(total)]
    for v in range(m):
        if matched[v] != -1:
            out.append(f"{matched[v] + 1} {v + 1}")
    sys.stdout.write("\n".join(out))

solve()
```

4.2 Emparejamiento Bipartito por Segmentación de Filas/Columnas

Preguntas guía:

- ¿La cuadrícula tiene obstáculos que "cortan" el alcance de piezas o rangos dentro de cada fila y columna?
- ¿Cada fila y cada columna se puede dividir en segmentos independientes (separados por obstáculos) donde a lo más un elemento puede colocarse por segmento?
- ¿Piden el máximo número de elementos colocables sin conflicto entre sí?

Utilidad: Resolver problemas de colocación de piezas sobre una cuadrícula con obstáculos, donde el conflicto es "compartir fila o columna sin un obstáculo de por medio". En vez de razonar celda por celda, se etiquetan los segmentos libres de cada fila y columna, y el problema se convierte en un emparejamiento bipartito clásico.

Complejidad: $O(N^2)$ para etiquetar segmentos, más $O(V \cdot E)$ para el emparejamiento (V, E acotados por N^2).

Fundamento teórico: Cada casilla vacía pertenece a exactamente un segmento de fila (corrida maximal de casillas vacías, delimitada por obstáculos o bordes) y a exactamente un segmento de columna. Como a lo más una torre puede ocupar cada segmento (todas sus casillas se atacan entre sí sin obstáculo de por medio), colocar torres sin conflicto equivale a elegir aristas entre segmentos de fila y segmentos de columna sin repetir segmento de ningún lado: exactamente un emparejamiento en el grafo bipartito (segmentos de fila) $\times$ (segmentos de columna), donde cada casilla vacía es una arista posible.

Ejercicio aplicado

Dado un tablero $N \times N$ con algunas casillas ocupadas por peones, hallar el máximo número de torres que se pueden colocar en casillas vacías sin que se ataquen entre sí (dos torres se atacan si comparten fila o columna sin un peón entre ellas). La entrada trae varios tableros seguidos.

I/O: $N \leq 100$, múltiples casos hasta EOF (patrón 4 de la sección de E/S) $\rightarrow$ lectura total con `sys.stdin.read().split()` y procesamiento por índice.

Código solución

```
import sys

def try_kuhn(u, adj, matched, visited):
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
```

```

        if matched[v] == -1 or try_kuhn(matched[v],
adj, matched, visited):
            matched[v] = u
            return True
    return False

def solve_board(n, grid):
    # etiqueta cada corrida maximal de celdas vacias en
    # cada FILA con un id
    row_id = [[-1] * n for _ in range(n)]
    row_count = 0
    for r in range(n):
        c = 0
        while c < n:
            if grid[r][c] == '.':
                while c < n and grid[r][c] == '.':
                    row_id[r][c] = row_count
                    c += 1
                    row_count += 1
            else:
                c += 1

    # lo mismo por COLUMNAS
    col_id = [[-1] * n for _ in range(n)]
    col_count = 0
    for c in range(n):
        r = 0
        while r < n:
            if grid[r][c] == '.':
                while r < n and grid[r][c] == '.':
                    col_id[r][c] = col_count
                    r += 1
                    col_count += 1
            else:
                r += 1

    # grafo bipartito: segmentos de fila (izq) --
    # segmentos de columna (der);
    # cada celda vacia es una arista posible entre ambos
    # segmentos
    adj = [[] for _ in range(row_count)]
    for r in range(n):
        for c in range(n):
            if grid[r][c] == '.':
                adj[row_id[r][c]].append(col_id[r][c])

    matched = [-1] * col_count
    total = 0
    for u in range(row_count):
        visited = [False] * col_count
        if try_kuhn(u, adj, matched, visited):
            total += 1
    return total

def main():
    data = sys.stdin.read().split()
    idx = 0
    out = []
    while idx < len(data):
        n = int(data[idx]); idx += 1
        grid = data[idx:idx + n]; idx += n
        out.append(str(solve_board(n, grid)))
    sys.stdout.write("\n".join(out) + "\n")

main()

```

4.3 Cobertura Mínima de Vértices en Grafos Bipartitos (Teorema de König)

Preguntas guía:

- ¿Piden el mínimo número de elementos a eliminar para que ningún conflicto (arista) quede activo?

- ¿El grafo de conflictos es bipartito (se puede partir en 2 grupos donde todo conflicto va de un grupo al otro)?
- ¿Ya sé calcular el emparejamiento máximo de ese grafo (entrada anterior, Kuhn)?

Utilidad: Calcular la cobertura mínima de vértices (mínimo conjunto de nodos que toca todas las aristas) en un grafo bipartito. En grafos generales este problema es NP-difícil, pero en grafos bipartitos se vuelve polinomial gracias al teorema de König.

Complejidad: $O(V \cdot E)$, la misma del emparejamiento bipartito máximo del que depende.

Fundamento teórico: El teorema de König establece que, en cualquier grafo bipartito, el tamaño de la cobertura mínima de vértices es EXACTAMENTE igual al tamaño del emparejamiento máximo. Basta con correr Kuhn (entrada anterior) y esa cifra ya es la respuesta. Importante: heurísticas locales (como quitar iterativamente puntos de articulación, o dividir el número de vértices con grado > 0 entre 2) NO son válidas para este problema en general - solo coinciden por casualidad en casos muy simples (aristas aisladas, caminos, estrellas) y fallan en cuanto el grafo tiene una estructura más rica (por ejemplo, un grafo bipartito 2-conexo sin puntos de articulación).

Ejercicio aplicado

Dado un tablero $N \times N$ con K caballos de ajedrez colocados, hallar el mínimo número de caballos que hay que quitar para que ninguno ataque a otro. El grafo de ataques es bipartito por construcción: cada movimiento de caballo cambia la paridad de (fila + columna), así que alcanza con particionar por esa paridad y correr Kuhn.

I/O: $N \leq 25$, $K \leq N^2 \leq 625 \rightarrow$ entrada pequeña, readline por línea (patrón 1).

Código solución

```

import sys
input = sys.stdin.buffer.readline

def try_kuhn(u, adj, matched, visited):
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            if matched[v] == -1 or try_kuhn(matched[v],
adj, matched, visited):
                matched[v] = u
                return True
    return False

def max_bipartite_matching(left_nodes, n_right, adj):
    matched = [-1] * n_right
    total = 0
    for u in left_nodes:
        visited = [False] * n_right
        if try_kuhn(u, adj, matched, visited):
            total += 1
    return total

def solve():
    n, k = map(int, input().split())
    pos = []
    for _ in range(k):
        r, c = map(int, input().split())
        pos.append((r - 1, c - 1))

```

```

index = {p: i for i, p in enumerate(pos)}
moves = [(1,2),(2,1),(-1,2),(-2,1),(1,-2),(2,-1),
         (-1,-2),(-2,-1)]

# el ataque de caballo SIEMPRE cambia el color (fila+
# col) par/impar:
# el grafo de ataques es bipartito por construccion,
# sin necesidad
# de demostrarlo -- solo hay que particionar por ese
# color
left_nodes = [i for i, (r, c) in enumerate(pos) if (r
+ c) % 2 == 0]
adj = [[] for _ in range(k)]
for i in left_nodes:
    r, c = pos[i]
    for dr, dc in moves:
        j = index.get((r + dr, c + dc))
        if j is not None:
            adj[i].append(j)

# Konig: cobertura minima de vertices ==
# emparejamiento maximo
print(max_bipartite_matching(left_nodes, k, adj))

solve()

```

4.4 Componentes Fuertemente Conexas (Algoritmo de Kosaraju)

Preguntas guía:

- ¿Es un grafo dirigido y preguntan por "grupos" donde todo par de nodos es mutuamente alcanzable?
- ¿Necesito "condensar" el grafo (contraer cada componente en un nodo) para trabajar luego sobre un DAG?
- ¿n puede ser grande ($\sim 10^5$), por lo que hay que evitar recursión profunda en Python?

Utilidad: Particionar un grafo dirigido en componentes fuertemente conexas (SCC): subconjuntos maximales donde cada nodo puede llegar a cualquier otro del mismo subconjunto. Es la base para condensar grafos, resolver 2-SAT y detectar ciclos mutuos.

Complejidad: $O(V + E)$ tiempo y espacio (dos recorridos DFS).

Fundamento teórico: Kosaraju usa dos pasadas de DFS: (1) en el grafo original, registrar el orden de finalización (postorden) de cada nodo; (2) en el grafo transpuesto (aristas invertidas), recorrer los nodos en orden inverso de finalización. Cada árbol de DFS resultante en esta segunda pasada es exactamente una SCC, porque invertir las aristas separa los componentes que antes solo eran alcanzables en una dirección.

Ejercicio aplicado

Un juego tiene n planetas conectados por m teletransportadores dirigidos. Dos planetas pertenecen al mismo "reino" si hay ruta en ambos sentidos entre ellos. Determinar, para cada planeta, a qué reino pertenece.

I/O: $n \leq 10^5$ y $m \leq 2 \cdot 10^5 \rightarrow$ lectura total + tokenización (patrón 2). Además, el DFS se implementa de forma iterativa (con pila explícita) para evitar RecursionError en Python con grafos grandes.

Código solución

```

import sys

def kosaraju(n, adj):
    visited = [False] * n
    order = [] # orden de finalizacion (postorden)
    for s in range(n):
        if visited[s]:
            continue
        visited[s] = True
        stack = [(s, iter(adj[s]))] # iterador
        # recuerda donde va en i parte de la lista
        while stack:
            node, it = stack[-1]
            advanced = False
            for nxt in it: # DFS iterativo
                con iteradores
                if not visited[nxt]:
                    visited[nxt] = True
                    stack.append((nxt, iter(adj[nxt])))
                    advanced = True
                    break
            if not advanced: # no quedan
                vecinos: cierra el nodo
                order.append(node)
                stack.pop()

    radj = [[] for _ in range(n)] # grafo
    # transpuesto (aristas invertidas)
    for u in range(n):
        for v in adj[u]:
            radj[v].append(u)

    comp = [0] * n
    visited = [False] * n
    c = 0
    for node in reversed(order): # orden inverso
        de finalizacion
        if visited[node]:
            continue
        c += 1
        visited[node] = True
        comp[node] = c
        stack = [node]
        while stack:
            u = stack.pop()
            for v in radj[u]:
                if not visited[v]:
                    visited[v] = True
                    comp[v] = c
                    stack.append(v)

    return c, comp

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); m = int(data[idx + 1]); idx += 2
    adj = [[] for _ in range(n)]
    for _ in range(m):
        a = int(data[idx]); b = int(data[idx + 1]); idx += 2
        adj[a - 1].append(b - 1)

    k, comp = kosaraju(n, adj)
    out = [str(k), " ".join(map(str, comp))]
    sys.stdout.write("\n".join(out))

solve()

```

4.5 Circuito Euleriano (Algoritmo de Hierholzer)

Preguntas guía:

- ¿Piden recorrer TODAS las aristas exactamente una vez (no todos los nodos, como en Hamiltoniano)?
- ¿Se pide un camino o circuito que use cada calle/arista una sola vez?
- ¿Puedo verificar rápido la condición de existencia (grados pares para circuito; 0 ó 2 nodos de grado impar para camino) antes de construir la ruta?

Utilidad: Encontrar un recorrido que use cada arista del grafo exactamente una vez (circuito o camino euleriano). Aplicaciones: rutas de reparto/barrido que cubren todas las calles, o cualquier problema de "recorrer cada arista una vez".

Complejidad: $O(V + E)$ con la versión iterativa basada en pila (evita recursión y eliminaciones costosas de aristas).

Fundamento teórico: Existe circuito euleriano si y solo si el grafo es conexo (en aristas) y todos los vértices tienen grado par. El algoritmo de Hierholzer avanza por aristas no usadas hasta atascarse; en ese momento inserta el vértice atascado en la ruta final y retrocede. El resultado, leído en el orden en que se van "atascando" los vértices, es un circuito válido.

Ejercicio aplicado

Dado un mapa de n cruces y m calles bidireccionales, hallar una ruta que empiece y termine en el cruce 1 (oficina postal) y recorra cada calle exactamente una vez, o determinar que es "IMPOSSIBLE".

I/O: $n \leq 10^5$, $m \leq 2 \cdot 10^5 \rightarrow$ lectura total + tokenización (patrón 2). Se evita `adj.erase()/remove()` ($O(n)$ por operación) usando un arreglo de punteros `ptr[]` que salta aristas usadas: así la eliminación es $O(1)$ amortizado.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); m = int(data[idx + 1]); idx += 2

    adj = [[] for _ in range(n + 1)]
    deg = [0] * (n + 1)
    for e in range(m):
        a = int(data[idx]); b = int(data[idx + 1]); idx += 2
        adj[a].append((b, e))
        adj[b].append((a, e))          # grafo no dirigido:
        # ambos sentidos (e es id de arista)
        deg[a] += 1
        deg[b] += 1

    if deg[1] == 0 or any(deg[v] % 2 for v in range(1, n + 1)):
        print("IMPOSSIBLE")
        return

    used = [False] * m
    ptr = [0] * (n + 1)              # guarda las ids de
    # las aristas
```

```
path = []
stack = [1]
while stack:
    v = stack[-1]
    while ptr[v] < len(adj[v]) and used[adj[v][ptr[v]]][1]:
        ptr[v] += 1                # salta aristas
    ya usadas (O(1) amortizado)
    if ptr[v] == len(adj[v]):      # reviso todas
        las aristas de v y no queda ninguna disponible
        path.append(v)
        stack.pop()
    else:
        u, eid = adj[v][ptr[v]]
        used[eid] = True
        stack.append(u)            # avanza por la
        # arista disponible

if len(path) - 1 != m:            # no se
    recorrieron todas las aristas
    print("IMPOSSIBLE")
    return

sys.stdout.write(" ".join(map(str, path)))

solve()
```

4.6 Puntos de Articulación (Algoritmo de Tarjan)

Preguntas guía:

- ¿Piden identificar nodos cuya eliminación desconecta el grafo (o aumenta el número de componentes)?
- ¿El grafo es no dirigido y se pregunta por robustez/conectividad ante fallos de un solo nodo?
- ¿ n puede ser grande, por lo que hay que evitar recursión profunda en Python?

Utilidad: Encontrar los vértices de corte (articulation points) de un grafo no dirigido: nodos que, al eliminarse, aumentan el número de componentes conexas. Es la base para analizar robustez de redes (qué nodo, si falla, parte el sistema en pedazos).

Complejidad: $O(V + E)$ con DFS iterativo (un solo recorrido con low-link).

Fundamento teórico: Se hace un DFS calculando, para cada nodo, su tiempo de descubrimiento (`disc`) y el menor tiempo de descubrimiento alcanzable usando a lo más una arista de retroceso (`low`). Un nodo u (no raíz) es punto de articulación si tiene un hijo v en el árbol DFS tal que $low[v] \geq disc[u]$: eso significa que desde el subárbol de v no hay forma de "escapar" hacia arriba de u sin pasar por u . La raíz del DFS es un caso especial: es punto de articulación si tiene más de un hijo en el árbol.

Ejercicio aplicado

Dadas varias pruebas, cada una con n islas y m puentes bidireccionales (grafo conexo), contar cuántas islas, al hundirse, desconectarían partes de la ciudad. La entrada termina con un caso 0 0.

I/O: Múltiples casos con centinela "0 0" (patrón 3 de la sección de E/S). $n \leq 10^4$, $m \leq 10^5 \rightarrow$ lectura total + tokenización, y DFS iterativo con pila explícita (mismo

motivo que en Kosaraju: evitar RecursionError/stack overflow).

Código solución

```
import sys

def find_articulation_points(n, adj):
    disc = [0] * n
    low = [0] * n
    visited = [False] * n
    is_ap = [False] * n
    timer = 1

    for start in range(n):
        if visited[start]:
            continue
        visited[start] = True
        disc[start] = low[start] = timer
        timer += 1
        root_children = 0
        # pila: [nodo, padre, iterador de vecinos,
        # ya_salto_arista_al_padre]
        stack = [[start, -1, iter(adj[start]), False]]
        while stack:
            frame = stack[-1]
            node, parent, it, _ = frame
            advanced = False
            for nxt in it:
                if nxt == parent and not frame[3]:
                    frame[3] = True # salta SOLO una
                    # vez la arista de vuelta
                    continue # (por si hay
                    # aristas paralelas)
                if not visited[nxt]:
                    visited[nxt] = True
                    disc[nxt] = low[nxt] = timer
                    timer += 1
                    if node == start:
                        root_children += 1
                    stack.append([nxt, node, iter(adj[nxt]), False])
                    advanced = True
                    break
            else:
                low[node] = min(low[node], disc[nxt])
                # arista de retroceso
                if not advanced:
                    stack.pop()
                if stack:
                    par = stack[-1]
                    par_node = par[0]
                    low[par_node] = min(low[par_node],
                    low[node])
                    # condicion clasica de punto de
                    # articulacion (no raiz)
                    if par_node != start and low[node] >=
                    disc[par_node]:
                        is_ap[par_node] = True
                    if root_children > 1: # caso especial: la
                    # raiz del DFS
                        is_ap[start] = True
        return is_ap

def main():
    data = sys.stdin.buffer.read().split()
    idx = 0
    out = []
    while True:
        n = int(data[idx]); m = int(data[idx + 1]); idx
        += 2
        if n == 0 and m == 0:
            break
```

```
adj = [[] for _ in range(n)]
for _ in range(m):
    u = int(data[idx]) - 1; v = int(data[idx +
1]) - 1; idx += 2
    adj[u].append(v)
    adj[v].append(u)
is_ap = find_articulation_points(n, adj)
out.append(str(sum(is_ap)))
sys.stdout.write("\n".join(out) + "\n")

main()
```

4.7 Algoritmo de Bron–Kerbosch (Cliques Máximos)

Preguntas guía:

- ¿Necesito encontrar un clique máximo o todos los cliques máximos de un grafo no dirigido?
- ¿El número de vértices es relativamente pequeño (aproximadamente $n \leq 50$)?
- ¿La solución requiere explorar todas las posibles extensiones de un conjunto de vértices compatible?

Utilidad:

Encontrar todos los cliques máximos o el clique máximo de un grafo no dirigido mediante backtracking con poda. Es la técnica estándar para resolver problemas de máxima compatibilidad, grupos completamente conectados y optimización sobre grafos pequeños.

Complejidad:

En el peor caso el algoritmo requiere

$$\mathcal{O}(3^{n/3})$$

que es óptimo para la enumeración de todos los cliques máximos.

El consumo de memoria es

$$\mathcal{O}(n).$$

En la práctica, el uso de pivote reduce considerablemente el número de llamadas recursivas.

Fundamento teórico:

Durante la búsqueda se mantienen tres conjuntos:

- R : vértices que pertenecen al clique actual.
- P : vértices que todavía pueden añadirse al clique.
- X : vértices ya procesados, utilizados para evitar generar soluciones repetidas.

Cuando tanto P como X quedan vacíos, el conjunto R constituye un clique máximo.

Para reducir la cantidad de ramas exploradas se selecciona un pivote u y únicamente se consideran los vértices de

$$P \setminus N(u),$$

donde $N(u)$ representa los vecinos del pivote. Esta optimización disminuye significativamente el árbol de búsqueda.

Ejercicio aplicado

Dado un grafo no dirigido, determinar el tamaño del mayor clique del grafo.

Cada vértice representa una persona y existe una arista cuando dos personas son compatibles. Se desea encontrar el grupo más grande donde todos los integrantes sean compatibles entre sí.

I/O:

Se reciben múltiples casos de prueba.

Cada caso especifica el número de vértices y aristas seguido por la lista de conexiones.

El algoritmo resulta adecuado para grafos pequeños, donde una búsqueda exhaustiva con poda es suficiente para obtener la solución óptima.

Código solución

```
def bronk(R, P, X):
    global best

    if not P and not X:
        best = max(best, len(R))
        return

    if P or X:
        u = next(iter(P | X))
    else:
        u = None

    if u is None:
        candidates = list(P)
    else:
        candidates = list(P - graph[u])

    for v in candidates:
        bronk(
            R | {v},
            P & graph[v],
            X & graph[v]
        )

        P.remove(v)
        X.add(v)

n, m = map(int, input().split())
graph = [set() for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    graph[u].add(v)
    graph[v].add(u)

best = 0
bronk(set(), set(range(n)), set())
print(best)
```

Observación:

El algoritmo de Bron-Kerbosch explora únicamente conjuntos que pueden convertirse en un clique válido. La versión con pivote es la utilizada habitualmente en programación competitiva debido a que reduce considerablemente el número de llamadas recursivas. Aunque su complejidad en el peor caso sigue siendo exponencial, resulta extremadamente eficiente para grafos pequeños, siendo la solución estándar para problemas de clique máximo en ICPC.

5 Búsqueda y Optimización

5.1 Búsqueda Binaria sobre la Respuesta + Two Pointers

Preguntas guía:

- ¿La respuesta es un número y existe una condición monótona (si X funciona, todo valor mayor/menor también funciona)?
- ¿Puedo verificar en $O(n)$ o $O(n \log n)$ si un valor candidato "cumple" la condición pedida?
- ¿Dentro de esa verificación necesito contar pares/elementos con una condición de suma sobre un arreglo ordenado? (ahí entra Two Pointers)

Utilidad: Cuando no es fácil calcular la respuesta directamente pero sí verificar si un candidato es válido, y esa validez es monótona, se "busca" la respuesta con búsqueda binaria sobre el rango de valores posibles. Two Pointers acelera el conteo en arreglos ordenados de $O(n^2)$ a $O(n)$.

Complejidad: Búsqueda binaria: $O(\log(\text{rango}))$ iteraciones. Cada verificación con Two Pointers: $O(n)$. Total: $O(n \log(\text{rango}))$.

Fundamento teórico: La búsqueda binaria sobre la respuesta funciona porque el conjunto de valores "válidos" forma un intervalo continuo (monotonía), reduciendo el espacio de búsqueda a la mitad en cada paso. Two Pointers explota que, en un arreglo ordenado, mover un extremo cambia la condición de suma de forma predecible, evitando recorrer todas las parejas.

Ejercicio aplicado

Dados N precios de categorías de joyas, un cliente que es el K -ésimo en la fila debe comprar dos joyas de categorías distintas sin repetir una combinación ya usada por otro cliente. Hallar el mínimo dinero que garantiza poder comprar: la K -ésima suma más pequeña entre todos los pares de categorías distintas.

I/O: N no está acotado explícitamente pero puede ser grande $\rightarrow$ lectura total + tokenización (patrón 2) para el arreglo de precios.

Código solución

```
import sys

def count_pairs_leq(prices, n, limit):
    # cuenta cuantos pares cumplen arr[i] + arr[j] <= limit
    # usando dos punteros sobre un arreglo YA ORDENADO
    count = 0
    left, right = 0, n - 1
    while left < right:
        if prices[left] + prices[right] <= limit:
            count += right - left  # todos los pares
            con "left" fijo
            left += 1
        else:
            right -= 1
    return count

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
```

```
prices = list(map(int, data[2:2 + n]))
prices.sort()

low = prices[0] + prices[1]  # menor suma posible
high = prices[n - 2] + prices[n - 1]  # mayor suma posible

while low < high:
    mid = low + (high - low) // 2
    if count_pairs_leq(prices, n, mid) >= k:
        high = mid  # mid cumple: intenta algo menor
```

```
else:
    low = mid + 1  # mid no cumple: sube el limite

print(low)

solve()
```

5.2 Enumeración por Máscaras de Bits (Bitmask Brute Force)

Preguntas guía:

- ¿Hay un pequeño número ($\leq \sim 20$) de decisiones binarias desconocidas (bits, elecciones sí/no) y hace falta probar combinaciones?
- ¿El espacio total de combinaciones (2^k) es lo bastante chico para probarlas todas dentro del límite de tiempo?
- ¿Cada combinación se puede armar y evaluar rápido a partir de su representación binaria?

Utilidad: Representar y recorrer exhaustivamente todas las configuraciones posibles de un conjunto pequeño de decisiones binarias. Cada máscara de bits codifica una asignación distinta de valores, permitiendo generar candidatos de forma sistemática y verificar cuál satisface la condición del problema.

Complejidad: $O(2^k \cdot \text{costo de evaluar una combinación})$, con k = número de incógnitas binarias.

Fundamento teórico: Cualquier asignación de k bits desconocidos corresponde biunívocamente a un entero entre 0 y $2^k - 1$. Recorriendo ese rango y extrayendo el bit j con $(\text{mascara} \gg j) \& 1$, se reconstruye cada combinación posible sin necesidad de recursión ni backtracking explícito. Es la enumeración más directa de un espacio de búsqueda combinatorio pequeño.

Ejercicio aplicado

Un mensaje M' y un divisor N' se recibieron con hasta 16 bits en total marcados como perdidos (*). Reconstruir un mensaje M compatible (M completando los '*' de M') tal que exista un N (completando los '*' de N') donde N divide a M exactamente.

I/O: $|M'| \leq 500$ pero el total de '*' (en M' y N' combinados) es $\leq 16 \rightarrow$ lectura total con readline es más que suficiente; el costo real está en las 2^{16} combinaciones, no en la entrada.

Código solución

```

import sys

input = sys.stdin.readline

def generate(strings):
    pos = [] # posiciones donde existen bits desconocidos
    for i, c in enumerate(strings):
        if c == '*':
            pos.append(i)
    k = len(pos)

    ans = []
    for mask in range(1 << k): # cada mascara representa una asignacion distinta de los k bits perdidos
        cur = strings[:]
        for j in range(k):
            if (mask >> j) & 1:
                cur[pos[j]] = '1'
            else:
                cur[pos[j]] = '0'
        ans.append(''.join(cur))
    return ans

M = list(input().strip())
N = list(input().strip())

messages = generate(M)
divisors = generate(N)

for m in messages: # probar todas las combinaciones posibles
    value_m = int(m, 2)
    for n in divisors:
        value_n = int(n, 2)

        if value_n == 0:
            continue # la division entre cero no existe
        if value_m % value_n == 0: # la primera combinacion valida es respuesta
            print(m)
            sys.exit()

```

5.3 Búsqueda Binaria sobre la Respuesta con Ventana de Bonificación

Preguntas guía:

- ¿Piden maximizar un mínimo (o minimizar un máximo) tras aplicar UNA modificación elegida de un conjunto de opciones?
- ¿Puedo verificar rápido, para un valor objetivo fijo, si ALGUNA elección lo logra?
- ¿La factibilidad es monótona: si un objetivo H es alcanzable, cualquier objetivo menor también lo es?

¿Para qué sirve? Resolver problemas de "maximizar el mínimo" cuando la modificación disponible tiene una forma estructurada (aquí, un refuerzo en forma de escalera), buscando binariamente el mejor valor objetivo y verificando factibilidad con una condición cerrada en $\mathcal{O}(n)$ por candidato.

Complejidad: $\mathcal{O}(n \log(\max x + K))$.

Fundamento teórico: Para un objetivo H fijo, una posición i con $x_i < H$ tiene una deficiencia $d_i = H - x_i$.

Si se refuerza en la posición p , la posición i recibe bonificación $K - (p - i)$ solo si $p - K + 1 \leq i \leq p$; para que alcance H , se necesita $p \leq i + K - d_i$. Además, ninguna posición fuera del rango de bonificación puede ser deficiente, lo que obliga a $p \geq R$ (la posición deficiente más a la derecha). Así, el conjunto de posiciones p válidas es la intersección de intervalos $[i, i + K - d_i]$ sobre todas las posiciones deficientes i ; como estos intervalos tienen extremo izquierdo creciente en i , la intersección es no vacía si y solo si $R \leq K + \min_i (i + x_i) - H$ (tomando el mínimo solo entre las posiciones deficientes). Esta condición se evalúa en $\mathcal{O}(n)$, y como la factibilidad de H es monótona (un refuerzo que logra mínimo H automáticamente logra cualquier $H' < H$), se puede buscar binariamente el mayor H factible.

Ejercicio aplicado

Dada una muralla de N segmentos con alturas iniciales, y un refuerzo que agrega K bloques al segmento elegido, $K - 1$ al anterior, y así sucesivamente, hallar la mayor altura mínima posible tras aplicar el refuerzo en un único segmento.

I/O: $N \leq 10^5 \rightarrow$ lectura total con tokenización (patrón 2). Verificado contra fuerza bruta en 300 casos aleatorios pequeños.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    x = list(map(int, data[2:2 + n]))

    def feasible(h):
        # deficientes: posiciones (1-indexadas) por debajo de h
        r = -1 # posicion deficiente mas a la derecha
        best = None # min(i + x[i]) entre las deficientes (1-indexado)
        for i in range(n):
            if x[i] < h:
                pos = i + 1
                r = pos
                val = pos + x[i]
                if best is None or val < best:
                    best = val
        if r == -1:
            return True # nadie por debajo de h: cualquier refuerzo sirve
        # interseccion de rangos validos de p: [pos, pos+K-d] para cada deficiente
        # no vacia <=> r <= K + best - h
        return r <= k + best - h

    lo, hi = 1, max(x) + k
    while lo < hi:
        mid = (lo + hi + 1) // 2 # busca el mayor H factible
        if feasible(mid):
            lo = mid
        else:
            hi = mid - 1
    print(lo)

solve()

```

6 Matemáticas y Simulación

6.1 Patrón de Reducción Binaria (tipo Josefo)

Preguntas guía:

- ¿El problema describe un proceso repetitivo/alternado (cada segundo elemento se "marca") sobre n elementos, con n muy grande ($\geq 10^9$)?
- ¿Simular paso a paso sería $O(n)$, demasiado lento, pero el patrón se repite de forma autosimilar al dividir por la mitad?
- ¿Puedo replantear el problema de tamaño n como un subproblema de tamaño $\sim n/2$ aplicando la misma regla?

Utilidad: Resolver en $O(\log n)$ problemas de simulación con patrones alternados sobre secuencias enormes, sin simular cada paso. Es la misma idea del clásico Problema de Josefo: en vez de eliminar uno por uno, se "salta" de una ronda completa a la mitad del problema restante.

Complejidad: $O(\log n)$ por consulta (cada ronda aprox. divide n a la mitad).

Fundamento teórico: En cada "ronda" del proceso, la mitad de los elementos son descartados/activados según la paridad de su posición. Reformulando la posición y el tamaño del problema tras cada ronda ($n' = n - \text{procesados}$, $i' = \text{nueva posición relativa}$) se llega a una recurrencia que converge en $O(\log n)$ pasos en vez de $O(n)$.

Ejercicio aplicado

Hay N personas en fila y un detector de metales defectuoso que se activa de forma alternada (la 1ª persona no lo activa, la 2ª sí, la 3ª no, ...; quien lo activa va al final de la fila). Dado que el coach está en la posición i , determinar en qué turno (entre quienes no activan el detector) pasará.

I/O: $T \leq 1000$ casos, cada uno con solo dos enteros $\rightarrow$ readline por línea (patrón 1) es suficiente y más simple.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def safe_position(n, i):
    # turno "seguro" (que no activa el patron) en que pasara
    # la posición i
    res = 0
    while i % 2 == 0:          # mientras i cae en la
        mitad "activada"      # cuenta los turnos seguros
        res += i // 2
        de esta ronda
        n -= i // 2            # reduce el problema a la
        mitad restante        # la posición i se
        i = n                  # convierte en el nuevo n
    res += (i + 1) // 2        # suma los turnos seguros
    de la ronda final
    return res

def solve():
    t = int(input())
    out = []
    for _ in range(t):
        n, i = map(int, input().split())
        out.append(str(safe_position(n, i)))
```

```
sys.stdout.write("\n".join(out))
```

```
solve()
```

6.2 Fuerza Bruta / Simulación Directa

Preguntas guía:

- ¿El tamaño de la entrada es pequeño ($N \leq 10^3$ o similar) y no hay una fórmula cerrada evidente?
- ¿Puedo calcular la respuesta iterando directamente sobre el espacio del problema sin que se dispare el tiempo?
- ¿El problema es más de matemáticas/probabilidad que de estructuras de datos?

Utilidad: Reconocer cuándo NO hace falta un algoritmo sofisticado: con restricciones pequeñas, iterar directamente (simulando la definición del problema) es la solución más simple, rápida de programar y menos propensa a errores. Identificar esto ahorra tiempo valioso en el contrarreloj.

Complejidad: Depende del problema; en este caso $O(n)$ con aritmética de punto flotante.

Fundamento teórico: Se calcula directamente la probabilidad de que NO haya coincidencia de cumpleaños entre k personas, multiplicando factores $(1 - (k-1)/n)$ sucesivamente (extensión del problema del cumpleaños), deteniéndose apenas esa probabilidad cae por debajo de 0.5 (la probabilidad de coincidencia supera el 50%).

Ejercicio aplicado

Dado N (número de días del año), hallar el mínimo número de personas necesarias en una sala para que la probabilidad de que dos compartan cumpleaños supere el 50%.

I/O: Entrada de un solo entero $\rightarrow$ readline (patrón 1), no requiere nada especial.

Código solución

```
import sys
input = sys.stdin.buffer.readline

n = int(input())
prob_no_coincidencia = 1.0
for k in range(1, n + 2):
    prob_no_coincidencia *= (n - (k - 1)) / n
    if prob_no_coincidencia < 0.5:
        print(k)
        break
```

6.3 Simulación Lineal (Máximo de Prefijo)

Preguntas guía:

- ¿La respuesta depende de un solo recorrido lineal del arreglo, calculando un máximo/mínimo/acumulado por posición?
- ¿No hay dependencias entre posiciones más allá de una comparación directa fila por fila?
- ¿El problema, aunque tenga una narrativa elaborada, se reduce matemáticamente a una fórmula simple tras identificar la variable clave?

Utilidad: Resolver directamente problemas donde, tras modelar bien la situación (aquí: el "cuello de botella" es la fila con menor espacio libre entre dos paredes), un solo recorrido $O(n)$ calculando un máximo/mínimo basta. Es clave en competencia reconocer cuándo un problema de apariencia compleja se reduce a esto.

Complejidad: $O(h)$ tiempo, $O(h)$ espacio (o $O(1)$ extra si se procesa en streaming).

Fundamento teórico: Las dos paredes se mueven a 1 m/s cada una. En la fila i , el espacio libre es $w - (l[i] + r[i])$ y se cierra a razón de 2 m/s (ambas paredes se acercan). El tiempo en que la fila i se cierra es $(w - l[i] - r[i]) / 2$. Las paredes chocan en la fila que se cierra primero: la de menor espacio libre inicial, equivalente al máximo de $(l[i] + r[i])$.

Ejercicio aplicado

Una sala de H filas y ancho W tiene una pared este y una pared oeste que se acercan a 1 m/s cada una desde que alguien entra. Cada fila i tiene un ancho ya ocupado por cada pared. Determinar en cuánto tiempo chocan las paredes.

I/O: $H \leq 10^5$, dos arreglos de tamaño $H \rightarrow$ lectura total + tokenización (patrón 2).

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    h = int(data[0]); w = int(data[1])
    left = list(map(int, data[2:2 + h]))
    right = list(map(int, data[2 + h:2 + 2 * h]))
    max_sum = max(left[i] + right[i] for i in range(h))
    ans = (w - max_sum) / 2
```

```
if ans.is_integer():
    # verifica parseo de float a
    int
    print(int(ans))
else:
    print(ans)
solve()
```

6.4 Simulación de Eventos con Colas

Preguntas guía:

- ¿Hay un recurso compartido con estado (dirección, disponibilidad) que cambia según el orden de llegada de eventos con timestamps?
- ¿Los eventos ya vienen ordenados (o se pueden ordenar) y conviene procesarlos con colas separadas por tipo?
- ¿El estado siguiente depende de comparar el próximo evento contra un tiempo límite (fin de uso actual del recurso)?

Utilidad: Simular un sistema con estado (aquí: una dirección de movimiento y un instante de fin de uso) que evoluciona a medida que llegan eventos de distintos tipos, usando una cola por tipo de evento en vez de simular minuto a minuto.

Complejidad: $O(n)$ - cada persona se encola y desencola una sola vez.

Fundamento teórico: Se mantienen dos colas (una por dirección deseada) y un estado (dirección activa, instante en que el recurso queda libre). En cada paso se mira el evento más próximo entre ambas colas: si coincide con la dirección activa, extiende el fin de uso; si es la dirección contraria, esa persona debe esperar (se cuenta pero no se procesa aún); si ya no hay eventos antes de que el recurso quede libre, se decide si invertir la dirección (si había gente esperando) o terminar ese ciclo de uso.

Ejercicio aplicado

N personas llegan a una escalera doble (10 segundos por cruce) en tiempos distintos, cada una queriendo ir en una de dos direcciones. Determinar el instante en que la última persona termina de usarla.

I/O: $n \leq 10^4 \rightarrow$ lectura total + tokenización (patrón 2); se usan dos deque (collections.deque) en vez de queue, ya que en Python deque tiene $O(1)$ real para popleft().

Código solución

```
import sys
from collections import deque

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); idx += 1
    left = deque() # personas que quieren ir en
    # direccion 0
    right = deque() # personas que quieren ir en
    # direccion 1
    for _ in range(n):
        t = int(data[idx]); d = int(data[idx + 1]); idx += 2
        (left if d == 0 else right).append(t)

    end = 0 # instante en que el recurso queda
    # libre
    ans = 0
    waiting = 0 # cuantos esperan en direccion
    # contraria a la activa
    direction = -1 # -1 = detenida

    while left or right or direction != -1:
        # mira el proximo evento disponible entre ambas
        # colas
        next_time, who = None, -1
        if left and (next_time is None or left[0] <
next_time):
            next_time, who = left[0], 0
        if right and (next_time is None or right[0] <
next_time):
            next_time, who = right[0], 1

        if direction == -1:
            # arranca detenida: la toma quien llegue
            # primero
            direction = who
            end = next_time + 10
            (left if who == 0 else right).popleft()
        elif who != -1 and next_time < end:
            if who == direction:
                end = max(end, next_time + 10) # se
                # sube mientras sigue en marcha
            else:
                waiting += 1 #
                # direccion contraria: debe esperar
```

```

        (left if who == 0 else right).popleft()
    else:
        # no hay mas eventos antes de que el recurso
        quede libre
        if waiting > 0:
            direction = 1 - direction # invierte
            sentido para atender a los que esperan
            end += 10
            waiting = 0
        else:
            ans = end
            direction = -1 # se detiene

    print(ans)
    solve()

```

6.5 Simulación XOR con Detección de Ciclos

Preguntas guía:

- ¿El estado del sistema se puede representar como un vector de bits, y cada operación disponible es un XOR con una máscara fija?
- ¿Las operaciones se aplican en un orden CÍCLICO fijo y predecible (no libre elección)?
- ¿Necesito detectar si el proceso entra en un ciclo infinito sin nunca alcanzar el estado objetivo?

¿Para qué sirve? Simular un proceso determinista donde el estado (aquí, qué bombillas están encendidas) cambia por XOR con máscaras fijas aplicadas en orden cíclico, deteniéndose al llegar al estado objetivo o detectando que nunca se alcanzará. Es aplicable a cualquier proceso de estado finito con transiciones deterministas.

Complejidad: $\mathcal{O}(N)$ en la práctica (ver fundamento teórico), a pesar de que el espacio de estados aparenta ser exponencial en M .

Fundamento teórico: Presionar los N interruptores en orden fijo $1..N$ corresponde a aplicar sucesivamente XOR con N máscaras fijas. Después de un CICLO COMPLETO (los N interruptores), el efecto neto sobre el estado es XOR con una constante fija $C = \text{XOR de todas las máscaras de los interruptores, sin importar el estado en que se esté}$. Como $v \oplus C \oplus C = v$, el estado al INICIO de cada ciclo solo puede tomar 2 valores posibles (el original, o el original $\oplus C$), alternando entre ambos. Esto acota el número de estados distintos alcanzables (y por lo tanto el número de pasos antes de repetir un estado o llegar a 0) a como mucho, $2N$: por eso la detección de ciclos con un conjunto de estados visitados nunca necesita más de un puñado de iteraciones, sin importar cuán grande sea M .

Ejercicio aplicado

Un conserje presiona N interruptores en orden cíclico $1, 2, \dots, N, 1, 2, \dots$. Cada interruptor alterna un subconjunto fijo de M bombillas. Dado el estado inicial de las bombillas encendidas, determinar cuántas veces hay que presionar interruptores hasta que TODAS las bombillas queden apagadas simultáneamente, o -1 si nunca ocurre.

I/O: $N, M \leq 1000 \rightarrow$ readline por línea (patrón 1) es más que suficiente, dado que el propio algoritmo termina

en a lo más $\sim 2N$ iteraciones (verificado: $< 0.03s$ incluso en el peor caso $N=M=1000$).

Código solución

```

import sys
input = sys.stdin.buffer.readline

n, m = map(int, input().split())
ldata = list(map(int, input().split())) # ldata[0]=
        cantidad, resto=bombillas encendidas

switch = []
for i in range(n):
    sdata = list(map(int, input().split()))
    tmp = 0
    for j in range(1, sdata[0] + 1):
        aux = 1 << (m - sdata[j]) # bit que
        # representa esa bombilla
        tmp += aux
    switch.append(tmp) # mascara XOR
    # de este interruptor

bulbs = 0
for j in range(1, ldata[0] + 1):
    aux = 1 << (m - ldata[j])
    bulbs += aux # estado
    # inicial como bitmask

idx = 0
res = 0
seen = set() # detecta si (
    # estado, interruptor actual) se repite
while True:
    if bulbs == 0:
        break # todas las
        # bombillas apagadas: listo
    if (bulbs, idx) in seen:
        res = -1 # ya visitamos
        # este estado exacto: nunca terminara
        break
    seen.add((bulbs, idx))
    info = switch[idx]

    bulbs ^= info # aplica el
    # interruptor actual

    if idx + 1 == n:
        idx = 0 # vuelve al
        # primer interruptor (orden ciclico)
    else:
        idx += 1
    res += 1

sys.stdout.write(str(res))

```

7 Construcción y Combinatoria

7.1 Algoritmos Constructivos: Invariantes y Paridad

Preguntas guía:

- ¿El problema pide construir explícitamente una configuración (o probar que es imposible), en vez de optimizar o contar?
- ¿Puedo idear una construcción simple (simetrías, intercambios, patrones repetidos) en vez de buscar entre todas las opciones?
- ¿La imposibilidad de otros valores se puede demostrar con un invariante (paridad, módulo, suma fija) que ninguna elección puede romper?

Utilidad: Resolver problemas donde no hace falta ningún algoritmo de búsqueda: basta con proponer una construcción explícita y justificar, con un argumento de invariante, que el resultado contrario (o cualquier otro valor puntual) nunca puede ocurrir sin importar qué elecciones se hagan.

Complejidad: $O(n)$ por caso - construir y emitir la respuesta.

Fundamento teórico: Se particiona $\{1, \dots, n\}$ en pares consecutivos $(2k-1, 2k)$ y se intercambian entre sí. Cada par contribuye al contador un valor relacionado con $V_k = (2k-1)(2k)$, que siempre es par; y ese aporte entra a la suma final como $-V_k$, 0 o V_k (las tres opciones son pares). Como la suma de términos pares es par, el resultado nunca puede ser impar - en particular, nunca puede ser 1.

Ejercicio aplicado

(Codeforces 2246A - farmpiggy and Subset Sum) Dado un entero par n , construir una permutación p de longitud n tal que, sin importar qué signo (sumar, restar o ignorar) se le asigne a cada término $i \cdot p_i$ al acumular un contador, el resultado final nunca pueda ser exactamente 1.

I/O: $t \leq 25$, $n \leq 50$ (par) $\rightarrow$ entrada pequeña, readline por línea (patrón 1) es suficiente.

Código solución

```
import sys
input = sys.stdin.buffer.readline

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    p = []
    for i in range(1, n + 1):
        # empareja (1,2), (3,4), ..., (n-1,n) e
        # intercambia cada pareja:
        # el aporte de cada pareja al contador siempre es
        # par (ver teoria)
        if i % 2 == 1:
            p.append(i + 1)      # posicion impar ->
            # apunta a la siguiente
        else:
            p.append(i - 1)      # posicion par -> apunta
            # a la anterior
    out.append(' '.join(map(str, p)))
sys.stdout.write('\n'.join(out) + '\n')
```

7.2 Algoritmos Constructivos: Construcción Explícita

Preguntas guía:

- ¿Piden construir explícitamente n valores con una relación de divisibilidad o proporción exacta entre ellos y su suma?
- ¿La condición se puede reescribir como una ecuación entre fracciones que deben sumar exactamente 1?
- ¿Existe una descomposición numérica conocida (como una fracción egipcia) que se pueda extender o escalar para cualquier n ?

Utilidad: Construir directamente arreglos o valores que cumplan una condición de divisibilidad o proporción exacta, usando descomposiciones numéricas conocidas en vez de búsquedas o heurísticas. Es frecuente en problemas "constructivos" donde la solución es una fórmula cerrada, no un algoritmo de recorrido.

Complejidad: $O(n)$ por caso.

Fundamento teórico: Si cada $a_i = S / k_i$ (S = suma total), la condición " S divisible por cada a_i " equivale a que los k_i sean enteros positivos distintos con $\sum(1/k_i) = 1$. Partiendo de la identidad $1 = 1/2 + 1/3 + 1/6$ y extendiendo el término $1/6$ con una cadena de potencias de 2, se obtiene una familia de k_i distintos para cualquier $n \geq 3$: la construcción cerrada $\{1, 2, 3, 6, 12, 24, \dots, 3 \cdot 2^{(n-3)}\}$. Los casos $n=1$ y $n=2$ se resuelven aparte (trivial e imposible, respectivamente).

Ejercicio aplicado

(Codeforces 2246B - ezraft and Array) Dado n , construir un arreglo de n enteros positivos distintos tal que la suma total sea divisible por cada uno de sus elementos, o determinar que no es posible.

I/O: $t \leq 50$, $n \leq 50 \rightarrow$ entrada pequeña, readline por línea (patrón 1).

Código solución

```
import sys
input = sys.stdin.buffer.readline

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    if n == 1:
        out.append('1')      # caso base: un
        # unico elemento
    elif n == 2:
        out.append('-1')      # a1/a2 y a2/a1
        # con ambos positivos
        # obliga a1 ==
        # a2 (no son distintos)
    else:
        # construccion cerrada a partir de 1 = 1/2 + 1/3
        # + 1/6:
        # {1, 2} mas una cadena de potencias de 2
        # escaladas por 3,
        # que nunca "chocan" con los divisores de 1 o 2
        vals = [1, 2] + [3 * (1 << k) for k in range(n - 2)]
        out.append(' '.join(map(str, vals)))
sys.stdout.write('\n'.join(out) + '\n')
```

7.3 Combinatoria: Conteo por Simetría de Binomios

Preguntas guía:

- ¿Piden contar subconjuntos o subsecuencias que cumplen una condición de suma o balance?
- ¿Hay valores repetidos en la entrada, de forma que agruparlos por valor simplifique el conteo?
- ¿Se puede usar $C(c, \text{par}) = C(c, \text{impar}) = 2^{c-1}$ para separar el conteo por grupo y combinar con una potencia de 2?

Utilidad: Contar eficientemente subconjuntos o subsecuencias que satisfacen una condición combinatoria, evitando enumerar los 2^n subconjuntos, aprovechando la simetría de los coeficientes binomiales para reducir el problema a una subsecuencia mucho más pequeña: solo los valores distintos.

Complejidad: $O(n)$ por caso (la entrada ya viene no-decreciente).

Fundamento teórico: Dentro de un grupo de c copias iguales, elegir una cantidad par de ellas no cambia la suma alternada acumulada (aporta 0) y elegir una cantidad impar sí la cambia (aporta $\pm \text{valor}$). Como $C(c, \text{par}) = C(c, \text{impar}) = 2^{c-1}$, cada grupo aporta el mismo peso sin importar el modo elegido, así que el problema se reduce a contar subconjuntos de suma alternada 0 sobre los valores DISTINTOS, multiplicando el resultado por 2^{n-m} al final (m = cantidad de valores distintos). En esa secuencia reducida, agrupando términos de dos en dos se demuestra que la única forma de obtener alternada 0 con más de un elemento es usando exactamente 3 valores: -1 y un par de enteros positivos consecutivos.

Ejercicio aplicado

(Codeforces 2246C - 0mar and Alternating Sums) Dado un arreglo no decreciente de n valores (cada uno -1 o un entero positivo), contar cuántas subsecuencias tienen suma alternada (primer término positivo, segundo negativo, tercero positivo, ...) igual a 0, módulo 10^9+7 .

I/O: $t \leq 10^4$, $\sum n \leq 2 \cdot 10^5 \rightarrow$ volumen moderado; se usa readline por línea (patrón 1), adecuado porque el costo dominante está en el procesamiento $O(n)$, no en la lectura.

Código solución

```
import sys
input = sys.stdin.buffer.readline

MOD = 10**9 + 7

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))

    # el arreglo ya viene no-decreciente: colapsar a
    # valores distintos.
    # cada grupo de c repetidos aporta un factor 2^(c-1)
    # independiente
    # del resto (C(c,par) == C(c,impar) == 2^(c-1))
    distinct = []
```

```
for x in a:
    if not distinct or distinct[-1] != x:
        distinct.append(x)

m = len(distinct)
has_neg1 = 1 if distinct[0] == -1 else 0 # -1 es el
    unico valor <= 0 posible

pos = [v for v in distinct if v > 0]
count_consec = 0
for i in range(len(pos) - 1):
    if pos[i + 1] - pos[i] == 1: # pares
        de enteros positivos consecutivos
        count_consec += 1

# en la secuencia reducida, la suma alternada solo
# puede dar 0 con:
# el subconjunto vacío, o exactamente {-1, v, v+1}
# para cada pareja
# consecutiva (si hay un -1 disponible)
reduced = (1 + (count_consec if has_neg1 else 0)) %
MOD
ans = reduced * pow(2, n - m, MOD) % MOD #
    reincorpora los repetidos
out.append(str(ans))

sys.stdout.write('\n'.join(out) + '\n')
```

7.4 Construcción Greedy por Alternancia (Límites de Racha Consecutiva)

Preguntas guía:

- ¿Hay que construir explícitamente una secuencia que respete un límite de repeticiones CONSECUTIVAS de cada elemento (sin límite en el total de usos)?
- ¿Un elemento puede reutilizarse libremente después de "descansar" al menos un turno?
- ¿Basta con dos "recursos" (el mejor y el segundo mejor) para romper cualquier racha, sin necesidad de planear la secuencia completa de antemano?

¿Para qué sirve? Construir una secuencia válida de longitud D usando N opciones con límites de racha consecutiva, sin necesidad de una búsqueda ni un algoritmo sofisticado: alternar entre las DOS opciones con mayor límite es siempre suficiente (si el problema es factible), porque cualquier racha se puede "romper" con un solo turno de la segunda opción antes de retomar la primera.

Complejidad: $O(D)$.

Fundamento teórico: El problema es imposible si ningún lugar admite ni una sola noche (todos los límites son 0), o si solo UN lugar tiene límite positivo y ese límite es menor que D (no hay forma de "descansar" de él). En cualquier otro caso (al menos dos lugares con límite > 0), siempre es posible: se usa el lugar de mayor límite hasta agotar su racha, se intercala un solo día en el segundo mejor lugar (lo cual reinicia la racha del primero), y se repite este patrón hasta completar los D días. Este patrón simple es suficiente porque no hay ningún límite en el número TOTAL de veces que se puede volver a un lugar, solo en la racha consecutiva.

Ejercicio aplicado

Kai necesita dormir D días seguidos, alternando entre N lugares, donde el lugar i tolera a lo más d_i noches CONSECUTIVAS antes de expulsarlo (pero puede volver después). Construir un plan válido día a día, o determinar que es imposible.

I/O: $N, D \leq 10^5 \rightarrow$ lectura total con tokenización (patrón 2). Verificado contra 200 casos aleatorios comparando factibilidad y validando la secuencia construida.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); d = int(data[1])
    arr = list(map(int, data[2:2 + n]))

    order = sorted(range(n), key=lambda i: -arr[i])
    best_idx = order[0]
    best_val = arr[best_idx]

    if best_val == 0:  # ningun lugar es utilizable
        print(-1)
        return

    second_val = arr[order[1]] if n > 1 else 0

    if second_val == 0:
        # solo hay UN lugar utilizable: hay que quedarse
        # ahí todos los D dias seguidos
        if best_val >= d:
            print(' '.join([str(best_idx + 1)] * d))
        else:
            print(-1)
        return

    second_idx = order[1]
    out = []
    remaining = d
    while remaining > 0:
        if remaining == 1:
            out.append(second_idx + 1)  # un día de "descanso" en el segundo mejor lugar
            break

        out.append(second_idx + 1)  # un día de "descanso" en el segundo mejor lugar
        remaining -= 1

        take = min(best_val, remaining)
        out.extend([best_idx + 1] * take)
        remaining -= take
    sys.stdout.write(' '.join(map(str, out)) + '\n')

solve()
```

7.5 Distribución de Días de Espera (Stars and Bars)

Preguntas guía:

- ¿Debo distribuir una cantidad fija de elementos idénticos entre varios lugares?
- ¿Cada lugar puede recibir cero o más elementos?
- ¿El orden dentro de cada grupo no importa, únicamente cuántos elementos recibe cada uno?

¿Para qué sirve?

Contar el número de formas de distribuir una cantidad fija de elementos indistinguibles entre varios grupos independientes. Es una de las aplicaciones más comunes de la técnica de *Stars and Bars* (estrellas y barras), utilizada frecuentemente en problemas de combinatoria, programación competitiva y teoría de conteo.

Complejidad:

Preprocesamiento de factoriales: $\mathcal{O}(MAX)$.

Cada consulta: $\mathcal{O}(1)$.

Espacio: $\mathcal{O}(MAX)$.

Fundamento teórico: Si el viaje mínimo hasta el oasis destino requiere O días y debe durar exactamente T días, existen $extra = T - O$ días de espera que pueden distribuirse libremente antes de salir o después de cualquiera de los D oasis recorridos, es decir, entre $D + 1$ posiciones distintas. Como los días de espera son indistinguibles y cada posición puede recibir cualquier cantidad de ellos, el problema se reduce directamente al principio de *Stars and Bars*, obteniendo un total de $\binom{extra+D}{D}$ planes posibles.

Ejercicio aplicado

Se conocen las posiciones de todos los oasis de una ruta.

Para cada consulta se recibe un oasis destino y el número exacto de días que debe durar el viaje.

Calcular cuántos planes distintos existen para distribuir los días de espera manteniendo siempre el mismo recorrido.

I/O:

$5 \leq N \leq 20$.

$1 \leq M \leq 10$.

Las distancias a los oasis no superan los 60 días, por lo que todos los factoriales necesarios pueden precomputarse una única vez antes de responder las consultas.

Cada consulta se responde en tiempo constante.

Código solución

```
import sys
input = sys.stdin.buffer.readline
cant, queries = map(int, input().split()) # numero de oasis y consultas
oasis = list(map(int, input().split())) # distancia minima hasta cada oasis

MAX = 80
factorial = [1] * (MAX + 1) # factoriales para responder combinaciones en O(1)

for i in range(1, MAX + 1):
    factorial[i] = factorial[i - 1] * i

def combination(n, k):
    return factorial[n] // (factorial[k] * factorial[n - k])

for _ in range(queries):
    destiny, time = map(int, input().split())
    extraTime = time - oasis[destiny - 1] # dias extra que deben distribuirse como esperas
    res = combination(extraTime + destiny, destiny) # Stars and Bars
    sys.stdout.write(str(res) + "\n")
```

8 Algoritmos de Cadenas (Strings)

8.1 Parejas Incompatibles en Secuencias de Paréntesis (Bloques de Nivel Superior)

Preguntas guía:

- ¿Trabajo con una secuencia balanceada de paréntesis y necesito razonar sobre qué emparejamientos específicos son válidos?
- ¿La pregunta involucra qué pasa con el RESTO de la secuencia al fijar/quitar un par específico?
- ¿Puedo identificar los "bloques de nivel superior" (tramos donde el balance nunca vuelve a cero hasta el final del bloque)?

¿Para qué sirve? Analizar, para una secuencia balanceada de paréntesis, la validez de emparejamientos NO necesariamente anidados de la forma estándar, descomponiendo la secuencia en bloques de nivel superior (tramos maximales donde el balance se mantiene estrictamente positivo hasta el cierre). La clave es que un '(' y un ')' se pueden emparejar sin romper la validez del resto si y solo si están en el MISMO bloque.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: Sea $\text{bal}(p)$ el balance (número de '(' menos número de ')') tras procesar los primeros p caracteres. Un bloque de nivel superior termina exactamente donde $\text{bal}(p) = 0$. Al emparejar una posición i '(' con una posición j ')', $i < j$, el balance de las posiciones ENTRE i y $j - 1$ baja en 1 respecto al original (porque ya se "usó" el '(' pero no el ')'); esto rompe la validez del resto si y solo si el balance ORIGINAL en algún punto de ese rango ya era 0 (pues bajaría a -1). Como los únicos puntos con balance 0 son los cierres de bloque, el par es INCOMPATIBLE si y solo si i y j pertenecen a bloques distintos. El total de parejas incompatibles se calcula entonces como (total de parejas (i, j) válidas en toda la secuencia) menos (parejas válidas contando solo dentro de cada bloque por separado). Ambas cantidades se obtienen en un único recorrido llevando un contador de '(' abiertos, global y reiniciado por bloque.

Ejercicio aplicado

Dada una secuencia de N bailarines Yang '(' y Yin ')' (garantizada balanceada), donde cada Yang debe emparejarse con un Yin posterior, contar cuántas parejas (i, j) son "incompatibles": aquellas que, de formarse, impiden que el resto de bailarines encuentre pareja.

I/O: $N \leq 10^6 \rightarrow$ un solo recorrido lineal con lectura total (patrón 2); no hace falta tokenizar por espacios ya que la secuencia es un único string contiguo. Verificado: ~0.1s en el peor caso.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    s = data[1].decode()
```

```
total_pairs = 0      # total de parejas (i,j) con i<j,
                      # s[i]='(', s[j]=')'
within_block = 0     # de esas, cuantas quedan DENTRO
                      # del mismo bloque de nivel superior
open_global = 0
open_block = 0
bal = 0
```

```
for ch in s:
    if ch == '(':
        open_global += 1
        open_block += 1
        bal += 1
    else:
        total_pairs += open_global
        within_block += open_block
        bal -= 1
        if bal == 0:      # cierre de un bloque
                           # de nivel superior y reinicia el contador de bloque
            open_block = 0
```

```
print(total_pairs - within_block)
```

solve()

8.2 Rotación Mínima de un String (Algoritmo de Booth)

Preguntas guía:

- ¿Dos strings se consideran "iguales" si uno es una rotación cíclica del otro?
- ¿Necesito una forma canónica única para comparar/deduplicar strings bajo rotación, en tiempo lineal (no $\mathcal{O}(n^2)$ probando todas las rotaciones)?
- ¿El string puede ser largo (miles de caracteres) y hay muchos para procesar?

¿Para qué sirve? Encontrar la rotación lexicográficamente menor de un string en $\mathcal{O}(n)$, como forma canónica para comparar o deduplicar strings equivalentes bajo rotación cíclica. Evita el enfoque ingenuo de generar y comparar las n rotaciones ($\mathcal{O}(n^2)$).

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: El algoritmo de Booth adapta la función de fallo (como en KMP) sobre el string duplicado $(s + s)$. Mantiene un índice candidato k a la rotación mínima y, al procesar cada carácter, usa la función de fallo para saltar comparaciones ya conocidas, actualizando k solo cuando encuentra evidencia de que otra posición produce una rotación menor. El resultado equivale a comparar todas las rotaciones en un solo recorrido lineal.

Ejercicio aplicado

Dado un conjunto de huellas dactilares codificadas como strings (equivalentes bajo rotación cíclica), eliminar duplicados representando cada grupo por su rotación canónica (la lexicográficamente menor).

I/O: Hasta 100 strings de hasta 5000 caracteres por caso, múltiples casos hasta el centinela "0" $\rightarrow$ lectura total con `sys.stdin.read().split()` (patrón 2). Verificado: ~1.4s para 5 casos $\times$ 100 strings $\times$ 5000 caracteres, y correcto contra fuerza bruta en 3000 strings aleatorios.

Código solución

```

import sys

def least_rotation(s):
    # indice de inicio de la rotacion lexicograficamente
    # menor (algoritmo de Booth)
    ss = s + s
    n = len(ss)
    fail = [-1] * n
    k = 0
    for j in range(1, n):
        sj = ss[j]
        i = fail[j - k - 1]
        while i != -1 and sj != ss[k + i + 1]:
            if sj < ss[k + i + 1]:
                k = j - i - 1
                i = fail[i]
            if sj != ss[k + i + 1]:
                if sj < ss[k + i + 1]:
                    k = j
                fail[j - k] = -1
            else:
                fail[j - k] = i + 1
    return k

def canonical(s):
    k = least_rotation(s)
    return s[k:] + s[:k]

def solve():
    data = sys.stdin.read().split()
    idx = 0
    out = []
    while idx < len(data):
        n = int(data[idx]); idx += 1
        if n == 0:
            break
        seen = set()
        for _ in range(n):
            m = int(data[idx]); f = data[idx + 1]; idx += 2
            c = canonical(f)
            if c not in seen:          # dedup
                # preservando orden de aparicion
                seen.add(c)
                out.append(c)
        sys.stdout.write("\n".join(out) + "\n")
    solve()

```

8.3 Validación de Matrioshkas Generalizadas (Pila con Restricción de Capacidad)

Preguntas guía:

- ¿La entrada representa una estructura jerárquica donde los elementos se abren y posteriormente deben cerrarse respetando el orden (LIFO)?
- ¿Cada elemento abierto posee una restricción que depende de todos los elementos contenidos directamente en su interior?
- ¿Necesito validar simultáneamente el correcto anidamiento y una condición adicional sobre el contenido de cada nivel?

¿Para qué sirve?

Validar estructuras anidadas donde cada objeto debe cerrarse exactamente en el orden inverso en que fue abierto y, además, debe cumplir una restricción sobre los

elementos contenidos directamente en su interior. Este patrón aparece frecuentemente en problemas de parsing, validación de expresiones jerárquicas y simulación de estructuras recursivas.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico:

Cada juguete comienza cuando aparece un entero negativo $-x$ y termina cuando aparece el correspondiente entero positivo x . Debido a que un juguete debe cerrarse antes de continuar cerrando el que lo contiene, el orden de cierre siempre sigue la política **Last In, First Out (LIFO)**, por lo que una pila modela naturalmente la estructura.

Ejercicio aplicado

Dada una secuencia de enteros que representa la apertura y cierre de juguetes generalizados, determinar si la estructura corresponde a una matrioshka válida.

Cada juguete de tamaño m puede contener varios juguetes, pero la suma de los tamaños de todos sus hijos directos debe ser estrictamente menor que m .

I/O:

Cada línea representa un caso de prueba independiente.

Los valores aparecen separados por espacios y pueden alcanzar magnitudes de hasta 10^7 . Como únicamente se realiza un recorrido secuencial sobre cada línea utilizando una pila, el algoritmo procesa cada caso en tiempo lineal.

Código solución

```

import sys

for line in sys.stdin:
    nums = list(map(int, line.split()))
    stack = []
    valid = True

    for x in nums:
        if x < 0:
            stack.append([-x, 0])
        else:
            if not stack:
                valid = False
                break
            size, total = stack.pop()      # numero del
            # cierre y cuantos tiene adentro
            if size != x:
                valid = False
                break
            if total >= size:
                valid = False
                break
            if stack:
                stack[-1][1] += size      # del ultimo en
            # en la cola le suma numero

    if stack:
        valid = False
    if valid:
        print(":-) Matrioshka!")
    else:
        print(":-( Try again.")

```

8.4 Normalización y Verificación de Palíndromos

Preguntas guía:

- ¿Piden verificar si un texto es un palíndromo ignorando mayúsculas, espacios y puntuación?
- ¿Basta con normalizar el string (filtrar solo los caracteres relevantes, unificar case) y comparar contra su reverso?
- ¿El caso de un string vacío (tras filtrar) debe considerarse un palíndromo válido?

¿Para qué sirve? Resolver directamente problemas de verificación de palíndromos con reglas de normalización, filtrando primero y comparando el resultado contra su versión invertida.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: Un string es palíndromo si es igual a su reverso. Al normalizar primero (quedarse solo con letras, todas en minúscula), la comparación directa `filtered == filtered[::-1]` captura exactamente la definición pedida, incluyendo el caso borde de que un string vacío (o sin letras) es trivialmente un palíndromo.

Ejercicio aplicado

Dada una oración, determinar si es un palíndromo ignorando mayúsculas, espacios, y cualquier carácter que no sea una letra del alfabeto inglés.

I/O: Múltiples líneas hasta el centinela "*" (patrón 3), $N < 1000$ por línea $\rightarrow$ se recorre línea por línea con `for line in sys.stdin`, ya que es texto libre (no se puede tokenizar con `.split()`).

Código solución

```
import sys

def solve():
    out = []
    for raw in sys.stdin:
        line = raw.rstrip('\n').rstrip('r')
        if line == '*':
            break
        filtered = [c.lower() for c in line if c.isalpha()]
        out.append('Y' if filtered == filtered[::-1] else 'N')
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

8.5 Eliminación de Ciclos en una Ruta (Stack Reduction)

Preguntas guía:

- ¿Debo procesar una secuencia donde ciertos elementos pueden cancelarse al aparecer nuevamente?
- ¿Eliminar un ciclo puede generar inmediatamente otro ciclo más grande?
- ¿Necesito conservar el orden original mientras simplifico la secuencia al máximo?

¿Para qué sirve?

Simplificar recorridos o secuencias eliminando ciclos innecesarios. Cada vez que un elemento vuelve a aparecer, todo el recorrido intermedio deja de ser útil y puede eliminarse. Después de cada eliminación es necesario volver a comprobar si ahora se formó un nuevo ciclo con los elementos restantes.

Este patrón aparece en problemas de rutas, navegación, directorios, historial de movimientos y simplificación de secuencias.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico:

La solución utiliza una pila que representa la ruta simplificada construida hasta el momento y cada máquina se procesa exactamente una vez y se inserta al final de la pila. Después de insertar una máquina pueden ocurrir dos situaciones. La primera es si las dos últimas máquinas tienen el mismo nombre donde la última visita es redundante y puede eliminarse. La segunda donde la última máquina coincide con la antepenúltima, por lo que tanto la máquina intermedia como la repetida pueden eliminarse.

Ejercicio aplicado

Dado un bang path de UUCP (UNIX-to-UNIX Copy Program), eliminar todos los saltos innecesarios producidos por ciclos entre máquinas, conservando exactamente el mismo destino final y respetando las mayúsculas y minúsculas de las máquinas que permanecen.

Las comparaciones entre nombres de máquinas son **case-insensitive**, aunque la salida debe conservar la capitalización original.

I/O:

La entrada consiste en una única línea cuya longitud no supera los 256 caracteres.

Cada componente está separado por el carácter '!' y el último elemento siempre corresponde al usuario destino, por lo que nunca participa en las simplificaciones.

Debido al pequeño tamaño de la entrada, basta dividir el string utilizando `split("!")` y recorrer los componentes una única vez.

Código solución

```
import sys
input = sys.stdin.buffer.readline
data = input().decode("utf-8").split("!") # Separa cada
componente del bang path

if len(data) <= 2: # Si no existen maquinas intermedias
    no hay nada que simplificar
    sys.stdout.write("!".join(data))
    exit()

machines = data[:-1] # todas las maquinas
user = data[-1] # usuario final (no se modifica)
stack = []

for machine in machines:
    stack.append(machine) # agrega la maquina actual
    a la ruta
    while True: # mientras aparezcan nuevos ciclos
        despues de eliminar otros
            if (len(stack) >= 2 and stack[-1].lower() ==
                stack[-2].lower()): #CASO AA
                stack.pop()
```

```
        continue

    if (len(stack) >= 3 and stack[-1].lower() ==
stack[-3].lower()): #CASO ABA
        stack.pop()      # elimina la ultima A
        stack.pop()      # elimina B
        continue
    break # ya no existen reducciones posibles
sys.stdout.write("!".join(stack + [user])) # reconstruye
el bang path conservando el usuario final
```